



SAPIENZA
UNIVERSITÀ DI ROMA

Automatic Design of Intelligent Systems: Applied to Chess

Ingegneria dell'Informazione, Informatica e Statistica

Corso di Laurea in Informatica

Laureando
Matt Clifford Trinidad Ramos

Relatore
Enrico Tronci



SAPIENZA
UNIVERSITÀ DI ROMA

Automatic Design of Intelligent Systems: Applied to Chess

INGEGNERIA DELL'INFORMAZIONE, INFORMATICA E STATISTICA
Informatica

Matt Clifford Trinidad Ramos

ID number 2048130

Advisor

Prof. Enrico Tronci

Academic Year 2025/2026

Automatic Design of Intelligent Systems:Applied to Chess
Sapienza University of Rome

© 2026 Matt Clifford Trinidad Ramos. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Author's email: ramos.2048130@studenti.uniroma1.it

Indice

1	Introduzione	1
2	Background	6
2.1	Gli scacchi come ambiente di simulazione controllato	6
2.2	Rappresentazione dello stato e generazione delle mosse legali	7
2.3	Controllori basati su reti neurali	8
2.4	Ottimizzazione black-box e Evolution Strategies	9
2.5	Metriche probabilistiche di prestazione	10
2.6	Stima sequenziale e regole di arresto	11
2.7	Statistical Model Checking	11
2.8	Il ciclo di apprendimento Quality-Guaranteed	12
3	Metodi	13
3.1	Panoramica del sistema	13
3.2	Formalizzazione di scenari, win-rate e requisiti	14
3.3	Loss di fattibilità	17
3.4	Training basato su Evolution Strategies	18
3.5	Verifica sequenziale per scenario	19
3.6	Statistical Model Checking della probabilità di violazione	20
3.7	Corrispondenza tra metodi e moduli Python	21
3.8	Aspetti non standard di questa istanziazione QG	22
3.9	Il ciclo Quality-Guaranteed completo	22
4	Implementazione	29
4.1	Panoramica dell'architettura software	29
4.2	Scelte ingegneristiche	30
4.3	Motore scacchistico	31
4.4	Codifica dello stato e rete neurale	32
4.5	Modulo Evolution Strategies	33
4.6	Modulo di verifica sequenziale	34
4.7	Motore di Statistical Model Checking	35
4.8	Gestione di scenari e controesempi	35
4.9	Logging, riproducibilità e strumenti	36

5	Risultati sperimentali	39
5.1	Setup sperimentale	39
5.1.1	Controllore e avversari	39
5.1.2	Scenari e requisiti	40
5.1.3	Budget di simulazione e criteri di arresto	40
5.2	Comportamento del training ES	41
5.2.1	Tassi di vittoria di training sull'insieme di scenari	43
5.3	Risultati della verifica scenario per scenario	43
5.3.1	Dimensione dei campioni e ampiezza degli intervalli	45
5.3.2	Decisioni pass/borderline/fail	45
5.4	Stime SMC della probabilità di violazione	46
5.4.1	Indicatori di violazione e intervalli di Wilson	46
5.4.2	Natura degli scenari di violazione	49
5.5	Budget complessivo di simulazione	49
5.6	Comportamento a livello di partita e confronto con le baseline	51
5.6.1	Partite contro avversari semplici	51
5.6.2	Partite contro baseline più forti	51
5.6.3	Statistiche quantitative a livello di partita	52
5.7	Sintesi dei risultati sperimentali	52
6	Conclusioni	55
6.1	Sintesi della tesi	55
6.2	Risposte alle domande di ricerca	56
6.3	Limiti del caso di studio	57
6.4	Prospettive e lavori futuri	58

Abstract

Questa tesi studia l'applicazione di un framework di apprendimento Quality-Guaranteed (QG) al dominio degli scacchi. L'obiettivo è progettare controllori che soddisfino un requisito di tasso di vittoria a livello di scenario, fornendo al contempo garanzie statistiche sulla probabilità di violazione in presenza di scenari incerti e precedentemente non osservati.

Il contributo principale di questo lavoro consiste nell'istanziamento di un ciclo QG che combina Evolution Strategies per l'ottimizzazione con Statistical Model Checking (SMC) per la verifica probabilistica. Il framework raffina iterativamente i controllori candidati minimizzando una feasibility loss definita a livello di scenario e stimando, tramite intervalli di confidenza, la probabilità di violazione associata alla soglia di prestazione richiesta.

Per ogni scenario campionato, un esperimento interno finito stima il tasso di vittoria a livello di scenario, che viene successivamente convertito in un indicatore binario di violazione. Una procedura di stima statistica sequenziale viene utilizzata per costruire intervalli di confidenza di Wilson e determinare se la probabilità di violazione sia inferiore a una tolleranza prefissata. Le garanzie risultanti sono di natura empirica e valgono con elevato livello di confidenza in regime di campione finito.

L'architettura proposta è implementata in un sistema modulare che integra componenti di ottimizzazione, verifica e gestione dei controesempi. I risultati sperimentali illustrano sia i punti di forza sia le limitazioni dell'applicazione di metodi quality-guaranteed ad ambienti di gioco complessi, evidenziando il compromesso tra prestazioni di ottimizzazione e affidabilità statistica.

Capitolo 1

Introduzione

I sistemi intelligenti sono sempre più impiegati in domini in cui le prestazioni devono essere non solo elevate, ma anche affidabili e certificabili. Dall'automazione industriale al processo decisionale autonomo, la sfida centrale non è più semplicemente *ottimizzare* un controllore, ma farlo *sotto un requisito di qualità esplicito* che possa essere misurato, testato e giustificato statisticamente. Le pipeline tradizionali di machine learning enfatizzano la prestazione media su dati di test separati; la pratica ingegneristica richiede invece enunciati del tipo: “il sistema ha successo con probabilità almeno τ e confidenza $1 - \delta$.” Colmare questo divario è l'obiettivo del controllo *Quality-Guaranteed (QG)*: un framework guidato dalla simulazione che integra ottimizzazione e verifica statistica per produrre garanzie quantitative sul comportamento di un sistema.

Questa tesi istanzia il framework QG in un dominio controllato ma non banale: il gioco degli **scacchi**. Gli scacchi offrono:

- dinamiche completamente specificate e deterministiche (mosse legali, stati terminali);
- uno spazio degli stati ricco, che mette in difficoltà politiche semplici e progettate a mano;
- un simulatore veloce e riproducibile, ideale per esperimenti Monte Carlo su larga scala.

Sebbene gli scacchi non comportino i rischi associati ai sistemi ciber-fisici, sono sufficientemente complessi da mettere alla prova la metodologia in un contesto avversariale e combinatorio. Più precisamente, il seed di scenario fissa la configurazione di alto livello (tipo di avversario e randomizzazione iniziale), mentre realizzazioni indipendenti sono ottenute consumando segmenti diversi dei flussi pseudo-casuali sottostanti (ad esempio scelte di esplorazione e tie-breaking), così che partite ripetute sotto lo stesso scenario possano essere modellate come prove di Bernoulli indipendenti, interpretate come un'approssimazione standard di tipo Monte Carlo piuttosto che come una garanzia formale di indipendenza.

Progettazione basata su simulazione con garanzie di qualità

Il paradigma della progettazione basata su simulazione assume che:

1. il sistema ad anello chiuso possa essere *simulato* per qualunque scelta dei parametri del controllore;
2. le esecuzioni possano essere *valutate* rispetto a un requisito di interesse;
3. le esecuzioni possano essere *ripetute* sotto randomizzazione controllata per stimare le probabilità rilevanti.

Su questo strato di simulazione, il controllo QG alterna due componenti:

- una **fase di ottimizzazione black-box** (nel nostro caso, Evolution Strategies) che cerca parametri del controllore θ minimizzando una loss di fattibilità guidata dal requisito;
- una **fase di Statistical Model Checking (SMC)** che stima, tramite intervalli di confidenza, quanto spesso il requisito venga violato sotto scenari randomizzati.

Queste due fasi formano un processo ad anello chiuso: se lo SMC rileva che il controllore viola il requisito troppo frequentemente, gli scenari che producono violazioni vengono trasformati in *controesempi* e aggiunti all'insieme di training, concentrando le successive fasi di ottimizzazione precisamente dove l'attuale controllore fallisce.

Perché le garanzie sono importanti

I sistemi di reinforcement learning basati su self-play possono raggiungere elevati win-rate empirici, ma senza limiti espliciti sulla probabilità di fallimento [10, 12]. In molte applicazioni, gli stakeholder non si accontentano di medie empiriche; richiedono invece enunciati del tipo

$$\Pr[\text{successo}] \geq \tau \quad \text{con confidenza } 1 - \delta,$$

oppure, in modo equivalente, ponendo $\varepsilon := 1 - \tau$,

$$\Pr[\text{fallimento}] \leq \varepsilon \quad \text{con confidenza } 1 - \delta.$$

L'approccio QG è pensato per questo tipo di enunciati, combinando l'apprendimento con evidenza probabilistica ottenuta tramite esperimenti Monte Carlo controllati.

Un semplice esempio numerico illustra l'idea. Se il requisito è “win-rate almeno 0.85”, una stima SMC potrebbe restituire

$$\hat{p} = 0.87, \quad [\ell, u] = [0.86, 0.90],$$

mostrando che, con la confidenza specificata, il win-rate reale supera la soglia.

Formulazione del problema nel dominio degli scacchi

L'istanziamento scacchistico della metodologia QG è composta da:

- **Dinamica del sistema.** Un motore scacchistico deterministico che agisce come ambiente: genera le mosse legali, aggiorna lo stato di gioco e determina gli esiti terminali.
- **Controllore.** Una rete neurale feed-forward leggera f_θ che valuta le posizioni e seleziona le mosse usando una politica greedy o ε -greedy.
- **Scenari.** Ogni scenario $s = (\sigma, o)$ fissa un seed casuale σ e una politica avversaria o . Gli scenari sono riproducibili e catturano la variabilità che il controllore deve gestire.
- **Requisito.** Una soglia di win-rate per scenario: il controllore deve soddisfarla per tutti gli scenari di training.

$$\text{WR}(\theta, s) \geq \tau$$

- **Loss di fattibilità.** Una loss centrata sul requisito $J_S(\theta)$, che aggrega di quanto ciascun scenario viola la soglia. Nel seguito della tesi, indichiamo il win-rate scenario per scenario con $p(\theta, s)$, e useremo $\text{WR}(\theta, s) \equiv p(\theta, s)$ in modo intercambiabile.

Il workflow QG in breve

Nel complesso, il ciclo QG applicato agli scacchi si articola in quattro passi:

1. **Ottimizza (ES).** Le Evolution Strategies minimizzano la loss di fattibilità sull'insieme di scenari S , producendo un controllore candidato θ^* .
2. **Verifica (SMC).** Lo Statistical Model Checking stima la probabilità di violazione

$$\gamma(\theta^*) = \Pr_{s \sim D} [p(\theta^*, s) < \tau_{\text{verify}}],$$

dove $p(\theta, s)$ denota il win-rate scenario per scenario e D è la distribuzione degli scenari non visti. Un intervallo di confidenza per $\gamma(\theta^*)$ è calcolato usando il metodo di Wilson.

3. **Focalizza (controesempi).** Tutti gli scenari che causano violazioni vengono aggiunti all'insieme di scenari S .
4. **Itera.** Il ciclo si ripete finché la probabilità di violazione stimata non scende al di sotto di una tolleranza prescritta $\varepsilon_{\text{viol}}$. In pratica, la procedura SMC stima un proxy empirico $\gamma_{n_{\text{int}}}(\theta^*)$, in cui il win-rate scenario per scenario è approssimato usando un numero finito n_{int} di partite interne per scenario.

Questa progettazione incarna il principio dell'*apprendimento sotto vincoli*: l'ottimizzazione è guidata da un requisito, non semplicemente dalla ricompensa media.

Ambito e assunzioni

L'ambito della tesi è delimitato dalle seguenti assunzioni di modellazione:

- **Classe di controllori.** Una rete feed-forward compatta, scelta per trasparenza, velocità e riproducibilità.
- **Distribuzione degli scenari.** Gli avversari includono politiche random e greedy-material, eventualmente con rumore; i seed inducono diversità nelle aperture e nelle tattiche di mediogioco.
- **Budget di simulazione.** Il numero di partite per scenario e il numero massimo di plies fungono da vincoli computazionali.
- **Interpretazione delle garanzie.** Tutte le garanzie si riferiscono alla distribuzione di scenari scelta; cambiare la distribuzione cambia l'enunciato che viene certificato.

Posizionamento rispetto alla letteratura esistente

La tesi si colloca all'intersezione tra decisione basata su reti neurali, ottimizzazione black-box e verifica statistica.

A differenza delle moderne pipeline di RL/self-play — che ottimizzano la prestazione attesa senza garanzie di confidenza — l'approccio proposto produce sia un controllore addestrato, sia un limite superiore quantitativo sulla sua probabilità di fallimento.

A differenza delle tecniche di controllo analitiche, il metodo sfrutta la simulazione per gestire la dinamica combinatoria degli scacchi.

Questa tesi fornisce un'istanza end-to-end del framework Quality-Guaranteed applicato a un simulatore di scacchi, integrando training, verifica e SMC in una pipeline unificata.

Motivazione

La progettazione di controllori affidabili con garanzie probabilistiche esplicite è sempre più importante nei contesti ad alta affidabilità. I metodi classici di reinforcement learning forniscono raramente tali garanzie, e i loro obiettivi di training non sono direttamente allineati con la soddisfazione di un requisito. Il framework QG affronta questo problema incorporando la verifica *all'interno* del processo di apprendimento, invece di applicarla solo *a posteriori*.

Domande di ricerca

La tesi indaga le seguenti domande:

- **RQ1.** Un controllore neurale leggero, addestrato tramite Evolution Strategies, può soddisfare un requisito di tipo must-win su un insieme finito di scenari?

- **RQ2.** La verifica sequenziale e lo Statistical Model Checking possono fornire garanzie significative controllate in confidenza entro budget di simulazione realistici?
- **RQ3.** Il raffinamento guidato da controesempi riduce la probabilità di violazione stimata e migliora la robustezza rispetto a scenari non visti?

Contributi

Questa tesi fornisce i seguenti contributi:

- **Istanziamento QG in un dominio discreto e avversariale.**
- **Training di fattibilità basato su ES** progettato per requisiti di win-rate non differenziabili.
- **Verifica sequenziale e SMC con garanzie a errore assoluto.**
- **Una pipeline software integrata, eseguibile solo su CPU**, che collega simulazione, ES, verifica e raffinamento tramite controesempi.
- **Uno studio empirico** che mostra il comportamento QG nel dominio degli scacchi.

Struttura della tesi

Il resto della tesi è organizzato come segue.

Il Capitolo 2 introduce l'ambiente scacchistico, i controllori neurali, l'ottimizzazione black-box e gli strumenti statistici utilizzati nel lavoro.

Il Capitolo 3 formalizza il ciclo di apprendimento QG.

Il Capitolo 4 descrive l'implementazione in Python.

Il Capitolo 5 presenta i risultati sperimentali.

Il Capitolo 6 riassume i risultati principali e delinea le possibili direzioni per lavori futuri.

Capitolo 2

Background

Questo capitolo raccoglie gli elementi teorici e metodologici necessari per il resto della tesi e li collega alla loro realizzazione concreta nella codebase Python. Per prima cosa introduciamo gli scacchi come ambiente di simulazione controllato e discutiamo come vengono rappresentati gli stati di gioco e le mosse legali. Descriviamo quindi il controllore basato su rete neurale, rivediamo le idee di base nell'ottimizzazione black-box e nelle Evolution Strategies (ES), e richiamiamo gli strumenti probabilistici utilizzati per valutare le prestazioni. Infine, presentiamo la stima sequenziale, lo Statistical Model Checking (SMC) e il ciclo di apprendimento Quality-Guaranteed (QG) a un livello informale, preparando il terreno per il trattamento formale nel Capitolo 3.

2.1 Gli scacchi come ambiente di simulazione controllato

Gli scacchi sono un gioco deterministico, a due giocatori, a turni, con informazione perfetta. In ogni momento della partita, lo stato completo è osservabile: la disposizione di tutti i pezzi sulla scacchiera, il colore cui spetta la mossa, i diritti di arrocco, eventuali catture *en passant* e i contatori delle mosse. Non c'è informazione nascosta e, una volta fissate le regole, non esiste alcuna sorgente esogena di randomizzazione.

Dal punto di vista della progettazione di controllori, è conveniente trattare gli scacchi come un *simulatore*. Dato uno stato corrente e una mossa scelta, il simulatore restituisce lo stato successore applicando le regole del gioco; dato uno stato, può determinare se la partita è terminata e, in tal caso, con quale esito (vittoria, patta o sconfitta). Questa prospettiva astrae dal gioco umano e si concentra sulla relazione di transizione di stato sottostante definita dalle regole.

Il caso di studio di questa tesi utilizza gli scacchi come banco di prova per la progettazione basata su simulazione con garanzie di qualità per tre motivi principali:

- **Dinamica deterministica.** Fissata una posizione iniziale e una sequenza di mosse, la partita risultante è univocamente determinata. Ciò rende la simulazione concettualmente semplice ed evita di dover modellare ambienti stocastici.

- **Ricca struttura combinatoria.** Il branching factor e la lunghezza delle partite generano uno spazio degli stati enorme, che mette in difficoltà controllori ingenui o progettati a mano.
- **Simulazione software veloce.** Un motore leggero può simulare un grande numero di partite in tempi ragionevoli su una CPU standard, aspetto cruciale per il ragionamento Monte Carlo e per la verifica QG.

L'implementazione in Python incapsula la logica del gioco in un modulo di motore dedicato. Internamente, ogni partita è rappresentata come una sequenza di stati e mosse, ma per la metodologia QG, nel calcolo dei win-rate e delle probabilità di violazione, è richiesto solo l'esito finale.

2.2 Rappresentazione dello stato e generazione delle mosse legali

Una *posizione* a scacchi codifica tutte le informazioni necessarie per proseguire la partita secondo le regole. Nell'implementazione, un oggetto posizione memorizza:

- l'occupazione di ciascuna casa della scacchiera 8×8 (tipo di pezzo o casa vuota);
- il lato cui spetta la mossa;
- i diritti di arrocco per entrambi i colori;
- un'eventuale colonna *en passant*;
- i contatori delle mezze mosse e delle mosse complete.

Sono possibili molte rappresentazioni interne (bitboard, array di pezzi, codifiche mailbox). Il progetto adotta una struttura semplice ma esplicita, che privilegia chiarezza e facilità di debug rispetto alle prestazioni pure. L'unico requisito imposto dal framework QG è che la rappresentazione sia deterministica e convertibile in modo efficiente nel vettore di feature consumato dal controllore neurale.

La generazione delle mosse legali segue una strategia standard in due passi:

1. **Mosse pseudo-legali.** Per ogni pezzo del lato di mossa, il motore genera anzitutto mosse che rispettano le regole di movimento specifiche del pezzo (torre, alfiere, cavallo, donna, re e pedone) ma ignorano temporaneamente la sicurezza del re.
2. **Filtro di legalità.** Per ogni mossa pseudo-legale, il motore simula la mossa su una copia della posizione e verifica se il re del giocatore che muove resterebbe sotto scacco. Solo le mosse che preservano la sicurezza del re vengono mantenute come legali.

Le regole speciali sono gestite in modo esplicito:

- **Arrocco** consentito solo quando re e torre non hanno mosso, le case intermedie sono vuote e il re non attraversa né raggiunge una casa sotto scacco.
- **Catture *en passant*** disponibili solo immediatamente dopo che un pedone avversario ha avanzato di due case e solo da pedoni adiacenti appropriati.
- **Promozioni** codificate come mosse che includono il tipo di pezzo promosso; tutte le opzioni di promozione legali vengono generate.

Le condizioni di terminazione (scacco matto, stallo, materiale insufficiente, ripetizione e regola delle cinquanta mosse) sono valutate dopo ogni mossa. Ciò garantisce che tutte le partite simulate terminino in tempo finito, sia raggiungendo una condizione terminale basata sulle regole sia toccando un numero massimo di plies specificato in configurazione.

2.3 Controllori basati su reti neurali

Il controllore considerato in questa tesi è una rete neurale che valuta posizioni scacchistiche e induce una politica per la selezione delle mosse. Formalmente, il controllore è una funzione

$$f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R},$$

dove $x \in \mathbb{R}^d$ è un vettore di feature che codifica la posizione corrente e θ raccoglie tutti i pesi e i bias addestrabili.

Lo stadio di codifica colma il divario tra la rappresentazione interna delle posizioni nel motore e l'input numerico atteso dalla rete. Una pipeline di codifica tipica è:

1. **Mappe binarie dei pezzi.** Per ciascun tipo di pezzo e colore, un piano binario 8×8 marca le case occupate da quel pezzo.
2. **Feature globali.** Componenti aggiuntive codificano il lato cui spetta la mossa, i diritti di arrocco e i contatori delle mosse.
3. **Flattening.** Tutti i piani e le feature scalari vengono concatenati in un vettore a dimensione fissata x .

La rete è un'architettura feed-forward compatta con uno o più strati nascosti e attivazioni non lineari (ad esempio ReLU). Il suo output scalare è interpretato come uno score per la posizione in input: valori più alti indicano posizioni che il controllore considera più promettenti per il lato di mossa.

Durante la simulazione delle partite, il controllore definisce una politica:

1. il motore elenca tutte le mosse legali dalla posizione corrente;
2. per ciascuna mossa, il motore la applica virtualmente per ottenere lo stato successore e codifica tale stato in x ;
3. il controllore calcola uno score $f_{\theta}(x)$ per ciascun successore;

4. viene selezionata la mossa con lo score più alto, eventualmente sostituita con una mossa casuale con probabilità ε (un meccanismo di esplorazione ε -greedy [12]).

Questa progettazione mantiene il controllore relativamente semplice, ma abbastanza espressivo da interagire in modo significativo con il ciclo QG e con l'ottimizzatore ES.

2.4 Ottimizzazione black-box e Evolution Strategies

Il framework QG tratta la progettazione del controllore come un problema di *ottimizzazione black-box*. Dato un vettore di parametri θ e un simulatore, possiamo valutare una funzione obiettivo $J(\theta)$ — nel nostro caso, una loss di fattibilità definita in termini di win-rate — ma non assumiamo di avere accesso a gradienti analitici o a una forma chiusa di J .

Le Evolution Strategies (ES) costituiscono una famiglia di algoritmi derivative-free adatti a questo tipo di contesti; si veda, ad esempio, [7] per un approccio moderno e scalabile applicato a contesti di reinforcement learning. Nella loro forma più semplice, le ES approssimano la discesa del gradiente sondando l'obiettivo intorno ai parametri correnti mediante perturbazioni casuali:

1. si campionano perturbazioni $\{\varepsilon_i\}_{i=1}^\lambda$ da una distribuzione normale multivariata $\mathcal{N}(0, \sigma^2 I)$;
2. si costruiscono parametri candidati $\theta_i = \theta + \varepsilon_i$;
3. si valuta l'obiettivo $J(\theta_i)$ per ciascun candidato tramite simulazione;
4. i candidati vengono ordinati o pesati in base ai rispettivi valori dell'obiettivo, dando più peso agli individui con loss più bassa;
5. si aggiornano i parametri di base come

$$\theta \leftarrow \theta + \alpha \sum_{i=1}^{\lambda} w_i \varepsilon_i,$$

dove α è un learning rate e w_i sono pesi di selezione normalizzati.

In valore atteso, questa procedura effettua un passo di gradiente stocastico su una versione smussata di J . Le ES sono particolarmente interessanti per questa tesi perché:

- possono ottimizzare obiettivi non differenziabili, come loss che coinvolgono parti positive o funzioni indicatrici;
- sfruttano naturalmente il parallelismo, poiché candidati e scenari differenti possono essere valutati in modo indipendente;
- trattano il simulatore come una black box, richiedendo solo valutazioni della funzione obiettivo.

Il Capitolo 3 costruisce su queste idee per definire la loss di fattibilità guidata dal requisito $J_S(\theta)$ e la relativa ottimizzazione basata su ES.

2.5 Metriche probabilistiche di prestazione

Poiché le simulazioni di scacchi sono deterministiche a seed e politiche fissati, la randomizzazione deriva dalla pseudo-randomizzazione e dal *campionamento degli scenari*. Intuitivamente, ogni scenario corrisponde a un seed fissato e a una politica avversaria fissata, e il controllore deve ottenere buone prestazioni su tali scenari.

Per un dato controllore θ e uno scenario s , l'esito di una singola partita è modellato come una variabile aleatoria di Bernoulli

$$Y(\theta, s) \in \{0, 1\},$$

dove $Y = 1$ denota una vittoria e $Y = 0$ denota una patta o una sconfitta (le patte sono conteggiate come fallimenti, così da imporre un vincolo di tipo must-win). La probabilità incognita di successo

$$p(\theta, s) = \Pr(Y(\theta, s) = 1)$$

rappresenta il *vero win-rate a livello di scenario*. La quantità $p(\theta, s)$ non è direttamente osservabile ed è stimata tramite simulazione Monte Carlo.

Se vengono giocate n partite indipendenti sotto lo stesso scenario, il win-rate empirico è

$$\hat{p}(\theta, s) = \frac{1}{n} \sum_{i=1}^n Y_i(\theta, s),$$

dove $Y_1(\theta, s), \dots, Y_n(\theta, s)$ sono trattate come realizzazioni i.i.d. di $Y(\theta, s)$. Sotto questa assunzione di modellazione, $\hat{p}(\theta, s)$ è uno stimatore non distorto di $p(\theta, s)$.

Sebbene le simulazioni di scacchi siano deterministiche a seed e politiche fissati, gli esiti di partite ripetute sotto lo stesso scenario sono trattati come variabili casuali. Realizzazioni indipendenti sono ottenute consumando porzioni diverse dei flussi pseudo-casuali sottostanti (ad esempio nelle scelte di esplorazione e nel tie-breaking), il che giustifica il modellare partite ripetute sotto uno scenario fissato come prove di Bernoulli i.i.d. Questa assunzione di indipendenza va intesa come un'astrazione di modellazione indotta dalla pseudo-randomizzazione, piuttosto che come una sorgente fisica di randomizzazione.

La metodologia QG utilizza queste quantità secondo due prospettive complementari:

- **Vista locale.** Su un insieme di training finito S di scenari, il requisito è che il vero win-rate a livello di scenario $p(\theta, s)$ superi una soglia τ per ogni $s \in S$.
- **Vista globale.** Sotto una distribuzione D sugli scenari, la quantità centrale è la *probabilità di violazione*

$$\gamma(\theta) = \Pr_{s \sim D}[p(\theta, s) < \tau],$$

cioè la probabilità che il requisito non sia soddisfatto.

La loss di fattibilità utilizzata durante il training ES aggrega, sull'insieme di training S , di quanto ciascun win-rate empirico $\hat{p}(\theta, s)$ scenda al di sotto della soglia τ . Le procedure di verifica associano poi intervalli di confidenza sia ai win-rate a livello di scenario sia alla probabilità di violazione globale.

2.6 Stima sequenziale e regole di arresto

Stimare una probabilità binomiale come $p(\theta, s)$ utilizzando un numero fisso e grande di campioni può risultare inefficiente quando la probabilità vera è molto alta o molto bassa. Le procedure *sequenziali* affrontano questo problema decidendo in modo adattivo quanti campioni raccogliere.

Per ciascuno scenario s , il verificatore mantiene il numero corrente di partite n_s , il numero di vittorie k_s e il win-rate empirico $\hat{p}_s = k_s/n_s$. Dopo ciascun batch di partite, viene calcolato un intervallo di confidenza $[\ell_s, u_s]$ per $p(\theta, s)$; il *raggio* dell'intervallo

$$r_s = \frac{u_s - \ell_s}{2}$$

fornisce una misura sintetica dell'incertezza statistica corrente.

L'implementazione utilizza gli intervalli score di Wilson per proporzioni binomiali, che garantiscono una buona copertura in campione finito e un comportamento stabile anche in prossimità delle probabilità 0 e 1 [14]. La stima sequenziale segue i principi classici dell'analisi sequenziale [13].

- Le partite vengono aggiunte in batch finché r_s non scende al di sotto di un'accuratezza target ε oppure finché non viene raggiunto un budget massimo $n_{\max}$.
- Una volta soddisfatta la condizione di arresto, lo scenario viene classificato come: *pass* se $\ell_s \geq \tau$, *fail* se $u_s < \tau$, oppure *borderline* negli altri casi (ossia quando $[\ell_s, u_s]$ interseca τ).

Questo approccio bilancia accuratezza statistica e costo di simulazione: scenari facili (con win-rate molto alto o molto basso) richiedono relativamente poche partite, mentre gli scenari *borderline* ricevono automaticamente maggiore attenzione. Nel ciclo QG, gli esiti *borderline/fail* sono trattati in modo conservativo come potenziali violazioni e possono essere usati per guidare il raffinamento tramite controesempi.

2.7 Statistical Model Checking

Il model checking classico mira a verificare se un sistema soddisfa una specifica formale per tutte le possibili esecuzioni. Per sistemi complessi o stocastici, l'esplorazione esaustiva dello spazio degli stati può risultare impraticabile. Lo Statistical Model Checking (SMC) sostituisce l'esplorazione esaustiva con il campionamento Monte Carlo: stima la probabilità che una proprietà sia soddisfatta osservando un numero finito di esecuzioni simulate; si veda, ad esempio, [1, 2, 9, 15].

In uno schema SMC tipico:

- si assumono un modello di sistema e una proprietà di interesse;
- si simulano esecuzioni indipendenti del sistema sotto condizioni campionate casualmente;
- ogni esecuzione è etichettata come *successo* o *fallimento* a seconda che la proprietà sia soddisfatta o meno;

- la frequenza empirica di successo è utilizzata per costruire stimatori e intervalli di confidenza per la probabilità vera di soddisfacimento.

In questa tesi, lo SMC è utilizzato in modo specializzato. La proprietà di interesse è un requisito di win-rate: “il controllore ottiene un win-rate a livello di scenario almeno pari a τ ”. Per ciascuno scenario estratto casualmente s , un piccolo esperimento interno produce una stima $\hat{p}(\theta, s)$ del win-rate a livello di scenario. Tale quantità viene poi convertita in un *indicatore di violazione* binario definito come

$$Z = \begin{cases} 1 & \text{se } \hat{p}(\theta, s) < \tau_{\text{verify}}, \\ 0 & \text{altrimenti.} \end{cases}$$

La sequenza di indicatori di violazione si comporta come un insieme di campioni Bernoulli i.i.d. con media

$$\gamma_{\text{int}}(\theta) = \Pr(\hat{p}(\theta, s) < \tau_{\text{verify}}),$$

che costituisce un'approssimazione empirica della vera probabilità di violazione $\gamma(\theta)$ definita nella Sezione 2.5. Lo SMC è quindi utilizzato per stimare $\gamma_{\text{int}}(\theta)$ con una garanzia di errore assoluto e un livello di confidenza prefissato.

2.8 Il ciclo di apprendimento Quality-Guaranteed

I concetti introdotti finora si combinano nel ciclo di apprendimento QG, un pattern di progettazione che accoppia ottimizzazione e verifica statistica ed è stato studiato in lavori precedenti su controllo e verifica QG [3].

A un livello alto, il ciclo QG alterna due fasi:

- **Fase di ottimizzazione.** Un controllore candidato è addestrato su un insieme finito di scenari S minimizzando una loss di fattibilità costruita a partire dai win-rate scenario per scenario. In questa tesi, le ES svolgono il ruolo di ottimizzatore, trattando il simulatore come una black box.
- **Fase di verifica.** Lo stesso controllore viene poi analizzato statisticamente: la verifica sequenziale per scenario raffina le stime di win-rate su S , mentre lo SMC stima la probabilità di violazione globale $\gamma(\theta)$ sotto una distribuzione di scenari più ampia.

Ogni volta che la verifica rivela scenari che violano il requisito, questi vengono registrati come *controesempi* e aggiunti all'insieme di training. Nel corso delle iterazioni, questo meccanismo di feedback arricchisce S con scenari critici e costringe il controllore a migliorare proprio dove attualmente fallisce.

Questo capitolo fornisce il background concettuale e probabilistico per questo paradigma. Il Capitolo 3 trasforma il ciclo QG in una formulazione matematica precisa, definisce in modo esplicito la loss di fattibilità e presenta la realizzazione algoritmica completa utilizzata nell'applicazione scacchistica. Le definizioni formali dei requisiti locali e globali, della loss di fattibilità e della probabilità di violazione sono fornite nelle Sezioni 3.2 e 3.3.

Capitolo 3

Metodi

Sulla base del quadro concettuale introdotto nel Capitolo 2, questo capitolo descrive come la metodologia Quality-Guaranteed (QG) venga istanziata nel caso di studio concreto degli scacchi implementato in Python. Più che presentare il controllo Quality-Guaranteed solo in termini astratti, l'obiettivo è collegare ogni oggetto matematico ai corrispondenti elementi della pipeline software: simulazione delle partite, stimatori di win-rate, Evolution Strategies (ES), verifica sequenziale e Statistical Model Checking (SMC). Tutti gli algoritmi presentati di seguito hanno controparti dirette nei moduli Python `chess_engine.py` e `qg_runner.py`, insieme ai moduli di ottimizzazione e verifica documentati nella Sezione 3.7. La struttura complessiva segue il paradigma generale della progettazione basata su simulazione con garanzie formali di qualità, adattato qui a un controllore scacchistico basato su rete neurale [3].

3.1 Panoramica del sistema

A livello alto, il sistema implementa un ciclo chiuso che gioca ripetutamente partite a scacchi tra un *controllore addestrabile* (il giocatore neurale da progettare) e una famiglia di politiche avversarie fissate. Tutti gli esperimenti sono guidati da seed pseudo-casuali, in modo che ogni simulazione sia riproducibile una volta noti il seed e il tipo di avversario.

I componenti principali sono:

- un **motore scacchistico** che mantiene gli stati di gioco e genera le mosse legali;
- un **controllore neurale** f_θ , implementato dalla classe `NNPlayer`, che valuta le posizioni e sceglie le mosse per il giocatore progettato;
- una **libreria di avversari** contenente politiche semplici e fissate, tra cui `Random-Player`, con l'elenco completo degli avversari riportato nella Sezione 3.7;
- uno **scenario manager**, che definisce e memorizza gli scenari usati sia per l'addestramento sia per la verifica;

- un **ottimizzatore ES** che esplora il vettore di parametri θ per ridurre una loss di fattibilità;
- uno **strato di verifica**, che fornisce verifica sequenziale per scenario e SMC globale.

Una singola partita a scacchi viene generata nel modo seguente. Lo scenario manager fornisce uno scenario $s = (\sigma, o)$, costituito da un seed σ e da una politica avversaria o . Il seed è usato per inizializzare tutti i generatori pseudo-casuali (ad esempio per l'esplorazione ε -greedy), mentre la politica avversaria descrive come l'altro giocatore seleziona le mosse. Il motore parte da una posizione iniziale fissata (o , in alcuni esperimenti, da una posizione di mediogioco predefinita) e alterna tra:

- interrogare il controllore quando è il turno del giocatore progettato;
- interrogare la politica avversaria quando è il turno dell'altro lato.

Lo stato viene aggiornato finché il motore rileva una condizione terminale (vittoria, patta o sconfitta). Il risultato finale è mappato in un *esito di partita* numerico, che viene poi aggregato su più partite per ottenere win-rate empirici.

La metodologia QG è incorporata in questo ciclo nel modo seguente:

1. dato un insieme finito di scenari di training S , l'ottimizzatore ES utilizza batch di partite simulate per minimizzare una *loss di fattibilità* $J_S(\theta)$;
2. quando ES è convergente (o dopo un budget fissato di iterazioni), lo strato di verifica stima i win-rate a livello di scenario e costruisce intervalli di confidenza tramite una procedura sequenziale basata sugli intervalli score di Wilson [14];
3. in parallelo, una procedura SMC stima la *probabilità di violazione* sotto una distribuzione di scenari D , ancora con garanzie di errore assoluto, seguendo i trattamenti standard SMC [1, 2, 9, 15];
4. gli scenari che presentano violazioni vengono memorizzati come controesempi e aggiunti a S , e il ciclo continua finché la probabilità di violazione stimata non scende al di sotto di una tolleranza prefissata.

L'Algoritmo 1 riassume come viene simulata una singola partita sotto un controllore θ e uno scenario s . Nell'implementazione, questa logica è realizzata dalle routine di simulazione delle partite in `chess_engine.py` ed è orchestrata da `qg_runner.py`.

Il resto del capitolo rende precisa questa descrizione: formalizziamo scenari e requisiti, spieghiamo come i win-rate sono modellati come quantità di tipo Bernoulli, definiamo la loss di fattibilità e descriviamo le procedure ES, di verifica e di SMC utilizzate nell'implementazione.

3.2 Formalizzazione di scenari, win-rate e requisiti

Iniziamo formalizzando come vengono calcolati i win-rate empirici sotto un controllore fissato e uno scenario fissato. Questa operazione di base è usata lungo tutto il training ES, nella verifica sequenziale e nello SMC.

1. *Selezione dello scenario.* Lo scenario manager campiona uno scenario $s = (\sigma, o)$ dall'insieme finito di training S oppure dalla distribuzione di scenari D (nel caso delle esecuzioni SMC).
2. *Simulazione della partita.* Data la coppia (θ, s) , il motore interroga ripetutamente il controllore e la politica avversaria per giocare una partita completa a scacchi, restituendo un esito Bernoulli $Y \in \{0, 1\}$ dal punto di vista del controllore.
3. *Stima del win-rate.* Ripetendo la simulazione della partita n volte sotto lo stesso scenario, il sistema calcola il win-rate empirico $\hat{p}(\theta, s)$ e, quando necessario, un intervallo di confidenza di Wilson per $p(\theta, s)$.
4. *Valutazione della loss (ES).* Durante il training, i win-rate empirici su tutti gli scenari in S sono aggregati nella loss di fattibilità $J_S(\theta)$, che guida l'aggiornamento ES.
5. *Verifica.* Una volta che ES è convergente, la verifica sequenziale per scenario affina le stime di $\hat{p}(\theta, s)$, mentre lo SMC stima la probabilità di violazione globale $\gamma(\theta)$ sotto D .
6. *Aggiornamento tramite controesempi.* Gli scenari che violano il requisito vengono memorizzati come controesempi e reinseriti in S , chiudendo il ciclo Quality-Guaranteed.

Figura 3.1. Pipeline logica del ciclo Quality-Guaranteed.

ID	Seed σ	Opponent policy o
s_1	123456789	Random policy
s_2	987654321	Greedy-material
s_3	4242424242	Greedy-material with noise
s_4	202520262027	Random policy (different opening bias)

Tabella 3.1. Esempi di scenari usati nel ciclo Quality-Guaranteed. Ogni scenario fissa un seed σ e una politica avversaria o , rendendo le simulazioni riproducibili per costruzione.

Scenari di esempio. Per rendere più concreta la definizione astratta di scenario, la Tabella 3.1 riporta alcuni esempi rappresentativi utilizzati negli esperimenti. Ogni scenario fissa sia un seed pseudo-casuale di base sia il tipo di avversario, rendendo la procedura sperimentale riproducibile. Le partite ripetute all'interno dello stesso scenario sono ottenute utilizzando seed casuali derivati differenti (ad esempio derivati in modo deterministico dal seed di base) oppure consumando porzioni diverse dei flussi pseudo-casuali sottostanti, in modo tale che gli esiti varino tra le ripetizioni pur rimanendo riproducibili.

Win-rate come quantità di Bernoulli

Dal punto di vista implementativo, una singola partita di scacchi è rappresentata da un oggetto match contenente l'identificativo dello scenario, la sequenza delle mosse e l'esito finale. Ai fini del controllo Quality-Guaranteed, è richiesto esclusivamente l'esito della partita. Ogni match è quindi modellato come una prova di Bernoulli dal punto di vista del giocatore progettato.

Fissati i parametri del controllore θ e uno scenario s , una singola partita genera una variabile aleatoria

$$\begin{aligned} Y_i(\theta, s) &\sim \text{Bernoulli}(p(\theta, s)), \\ \Pr(Y_i(\theta, s) = 1) &= p(\theta, s), \\ \Pr(Y_i(\theta, s) = 0) &= 1 - p(\theta, s). \end{aligned}$$

Nell'implementazione, le patte sono trattate come fallimenti al fine di imporre un requisito di tipo must-win. La probabilità di successo sottostante, ignota a priori,

$$p(\theta, s) = \Pr(Y(\theta, s) = 1)$$

rappresenta il *win-rate a livello di scenario*. Nel codice, $p(\theta, s)$ non viene mai calcolata in forma analitica; il simulatore genera invece esiti ripetuti di partite sotto la coppia (θ, s) , registrandoli come una sequenza di 0 e 1. Sebbene le dinamiche di gioco siano deterministiche a seed e politiche fissati, l'indipendenza tra le partite è approssimata eseguendo match ripetuti con seed pseudo-casuali derivati diversi, pur mantenendo invariata la definizione dello scenario.

Se vengono giocate n partite nello stesso scenario s , il win-rate empirico

$$\hat{p}(\theta, s) = \frac{1}{n} \sum_{i=1}^n Y_i(\theta, s)$$

è implementato semplicemente come la media degli esiti osservati. Sotto l'assunzione ideale che gli esiti $\{Y_i(\theta, s)\}_{i=1}^n$ siano prove di Bernoulli indipendenti con parametro $p(\theta, s)$, questa media campionaria è uno stimatore non distorto di $p(\theta, s)$. Nella nostra implementazione, l'indipendenza è soltanto approssimata tramite seed pseudo-casuali derivati; per questo usiamo $\hat{p}(\theta, s)$ come uno stimatore empirico standard e ne quantifichiamo l'incertezza tramite intervalli di confidenza binomiali.

Scenari e requisiti

Uno *scenario* è definito come una coppia

$$s = (\sigma, o),$$

dove σ è un seed intero a 64 bit e o è un identificatore di politica avversaria. Nel software, gli scenari sono memorizzati come piccoli record che contengono il seed e un'enumerazione che indica la politica avversaria (ad esempio random, greedy o greedy-with-noise). Fissare s determina sia la randomizzazione del controllore (tramite il seed) sia il comportamento dell'avversario.

Si considerano due livelli di requisito:

- un **requisito di training** su un insieme finito di training S , che richiede che il win-rate superi una soglia τ_{train} scenario per scenario;
- un **requisito di verifica** espresso in termini di probabilità di violazione sotto una distribuzione D sugli scenari, definito usando una soglia (eventualmente più severa) τ_{verify} .

Formalmente, il requisito di training è

$$p(\theta, s) \geq \tau_{\text{train}} \quad \text{per ogni } s \in S, \quad (3.1)$$

La proprietà globale è espressa in termini di *probabilità di violazione*

$$\gamma(\theta) = \Pr_{s \sim D} [p(\theta, s) < \tau_{\text{verify}}], \quad (3.2)$$

dove D è una distribuzione di probabilità sullo spazio degli scenari (in pratica, una distribuzione uniforme sui seed combinata con una distribuzione discreta sui tipi di avversario).

In questo caso di studio usiamo $\tau_{\text{train}} = 0.5$ come guida all'ottimizzazione basata su ES e $\tau_{\text{verify}} = 0.6$ per la verifica sequenziale e la valutazione della generalizzazione tramite SMC.

Probabilità di violazione empirica. In pratica il vero win-rate $p(\theta, s)$ è ignoto e può essere stimato solo tramite un numero finito di partite simulate. Per lo SMC, per ciascuno scenario campionato s calcoliamo un win-rate empirico $\hat{p}_{n_{\text{int}}}(\theta, s)$ basato su n_{int} partite interne.

Di conseguenza, lo SMC stima la probabilità di violazione empirica

$$\gamma_{n_{\text{int}}}(\theta) = \Pr_{s \sim D} [\hat{p}_{n_{\text{int}}}(\theta, s) < \tau_{\text{verify}}]. \quad (3.3)$$

Il requisito QG è che

$$\gamma(\theta) \leq \varepsilon_{\text{viol}},$$

per una piccola tolleranza $\varepsilon_{\text{viol}}$ (ad esempio, 0.05), con garanzie esplicite (ε, δ) sull'accuratezza degli stimatori usati sia per le quantità locali sia per quelle globali. I valori numerici specifici adottati negli esperimenti sono riportati nel Capitolo 5.

3.3 Loss di fattibilità

L'addestramento è guidato da una loss di fattibilità che penalizza gli scostamenti rispetto al requisito definito scenario per scenario. Per ogni $s \in S$, il win-rate empirico $\hat{p}(\theta, s)$ è stimato giocando un numero fissato di partite (specificato in configurazione) con parametri del controllore θ . La loss è definita come

$$J_S(\theta) = \sum_{s \in S} \max(0, \tau_{\text{train}} - \hat{p}(\theta, s)). \quad (3.4)$$

L'Algoritmo 3 corrisponde alle routine di calcolo della loss usate durante il training ES, implementate in `es_bbo.py` e richiamate da script driver come `run_es_once.py` e `qg_runner.py`.

Per costruzione, $J_S(\theta) = 0$ se e solo se il win-rate stimato raggiunge o supera τ_{train} su ogni scenario in S . L'implementazione calcola (3.4) direttamente dai risultati di simulazione: per ogni scenario aggrega gli esiti delle partite, calcola il win-rate empirico e aggiunge al totale la corrispondente parte positiva $\max(0, \tau_{\text{train}} - \hat{p}(\theta, s))$.

Questa loss è non differenziabile e rumorosa, perché i win-rate empirici sono basati su un numero finito di partite. Queste proprietà motivano l'uso di un metodo di ottimizzazione black-box, discusso nella sezione successiva.

3.4 Training basato su Evolution Strategies

L'ottimizzatore ES opera sulla loss di fattibilità (3.4). Il vettore di parametri θ raccoglie tutti i pesi e i bias della rete neurale di controllo; la sua dimensione dipende dall'architettura scelta ed è fissata una volta specificata l'architettura.

Ogni iterazione ES procede come segue.

1. **Population sampling.** Si campiona una popolazione di λ perturbazioni $\{\varepsilon_i\}_{i=1}^\lambda$ da una distribuzione normale multivariata $\mathcal{N}(0, \sigma^2 I)$. I parametri candidati sono costruiti come $\theta_i = \theta + \varepsilon_i$.
2. **Loss evaluation.** Per ciascun θ_i , il simulatore è invocato per stimare $\hat{p}_s(\theta_i)$ su tutti gli scenari $s \in S$, usando un numero fissato di partite per scenario. L'implementazione esegue queste partite in parallelo sulle diverse CPU, così che individui e scenari differenti possano essere valutati contemporaneamente. La loss di fattibilità $J_S(\theta_i)$ è poi calcolata tramite (3.4).
3. **Weight construction.** Le loss $J_S(\theta_i)$ sono convertite in pesi w_i ordinando gli individui e assegnando pesi di selezione maggiori ai candidati con loss più bassa. I pesi sono normalizzati in modo che $\sum_i w_i = 1$.
4. **Parameter update.** Il vettore medio dei parametri viene aggiornato secondo

$$\theta \leftarrow \theta + \alpha \sum_{i=1}^{\lambda} w_i \varepsilon_i, \quad (3.5)$$

dove $\alpha > 0$ è un learning rate specificato in configurazione.

L'aggiornamento dei parametri può essere interpretato come un passo di ricombinazione pesata basata sul ranking, che sposta il vettore dei parametri nelle direzioni associate alle perturbazioni migliori.

Dati i candidati campionati $\theta_i = \theta + \varepsilon_i$, con $\varepsilon_i \sim \mathcal{N}(0, \sigma^2 I)$, e pesi normalizzati $w_i \geq 0$, $\sum_{i=1}^{\lambda} w_i = 1$, assegnati in ordine decrescente di performance (cioè pesi maggiori ai candidati con loss minore), la regola di aggiornamento

$$\theta \leftarrow \theta + \alpha \sum_{i=1}^{\lambda} w_i \varepsilon_i$$

corrisponde a un passo di ricerca stocastica che sposta il parametro medio verso direzioni associate a migliori prestazioni empiriche. Questa interpretazione è coerente con le Evolution Strategies basate su rango e con i meccanismi di ricombinazione pesata comunemente usati nelle varianti ES e CMA-ES [6, 7, 8]. Nel software, questa logica è incapsulata nella classe `EvolutionStrategiesBBO` in `es_bbo.py`, che è guidata da script come `run_es_once.py` e dall'orchestratore QG `qg_runner.py`.

La fase di training ES termina quando si raggiunge un numero massimo di iterazioni oppure quando la loss empirica smette di diminuire in modo significativo. Il valore finale di θ viene quindi passato allo strato di verifica.

3.5 Verifica sequenziale per scenario

Mentre ES utilizza stime relativamente grossolane di $\hat{p}(\theta, s)$ (poche partite per scenario) per mantenere l'addestramento economicamente sostenibile, lo strato di verifica mira a fornire informazioni accurate a livello di scenario con controllo esplicito dell'errore. Per ogni $s \in S$, l'implementazione esegue una procedura sequenziale che decide in modo adattivo quante partite giocare.

L'Algoritmo 5 rispecchia l'implementazione in `sequential_verification.py`.

La verifica per scenario viene eseguita richiamando ripetutamente il simulatore di partite e aggiornando gli intervalli di Wilson finché la regola di arresto non è soddisfatta.

Intervalli score di Wilson nella pratica

Sia n_s il numero di partite giocate finora sotto lo scenario s , e sia k_s il numero di vittorie. Il win-rate empirico sotto il controlllore θ è

$$\hat{p}(\theta, s) = \frac{k_s}{n_s}.$$

Per quantificare l'incertezza, l'implementazione utilizza l'intervallo score di Wilson per proporzioni binomiali [14]. Dato un livello di confidenza $1 - \delta$ e il corrispondente quantile normale $z = z_{1-\delta/2}$, l'intervallo di Wilson è calcolato come

$$[\ell_s, u_s] = \frac{\hat{p}(\theta, s) + \frac{z^2}{2n_s}}{1 + \frac{z^2}{n_s}} \pm \frac{z \sqrt{\frac{\hat{p}(\theta, s)(1 - \hat{p}(\theta, s))}{n_s} + \frac{z^2}{4n_s^2}}}{1 + \frac{z^2}{n_s}}. \quad (3.6)$$

Nel codice, questa formula è implementata in una funzione di utilità dedicata, che prende in input la coppia (k_s, n_s) e il livello di confidenza e restituisce (ℓ_s, u_s) .

Sarebbe stato possibile utilizzare diversi altri intervalli di confidenza (per esempio, l'intervallo “esatto” di Clopper–Pearson o intervalli bayesiani basati su una prior di Jeffreys), ma l'intervallo di Wilson offre un compromesso favorevole tra accuratezza e semplicità. Fornisce una buona copertura anche per dimensioni di campione moderate e probabilità vicine a 0 o 1, rimanendo economico da calcolare e bilanciato nel trattamento di win-rate bassi e alti. Queste proprietà lo rendono ben adatto al grande numero di stime binomiali richieste dal ciclo QG.

Regola di arresto sequenziale

Il verificatore sequenziale per un singolo scenario opera a round successivi:

1. inizializza $n_s = 0$, $k_s = 0$;
2. gioca un nuovo batch di partite sotto lo scenario s con il controlllore finale θ , aggiornando i conteggi (k_s, n_s) ;
3. ricalcola $\hat{p}_s$ e l'intervallo di Wilson $[\ell_s, u_s]$;

4. verifica se il raggio dell'intervallo

$$r_s = \frac{u_s - \ell_s}{2}$$

è inferiore a un'accuratezza prescritta ε , oppure se è stato raggiunto un numero massimo di partite.

Se la condizione di arresto è soddisfatta, lo scenario viene classificato secondo le seguenti regole:

- *pass* se $\ell_s \geq \tau_{\text{verify}}$;
- *fail* se $u_s < \tau_{\text{verify}}$;
- *borderline* altrimenti (trattato di default come fail negli esperimenti).

Il verificatore restituisce, per ogni s , la stima finale $\hat{p}_s$, i limiti dell'intervallo, il numero di partite utilizzate e la decisione pass/fail. Queste informazioni vengono poi usate sia nel capitolo sperimentale sia nel raffinamento dell'insieme di scenari guidato dai controesempi.

3.6 Statistical Model Checking della probabilità di violazione

La verifica per scenario controlla le prestazioni sull'insieme finito S ma non fornisce, da sola, garanzie sulla probabilità di violazione globale $\gamma(\theta)$ in (3.2). Nell'implementazione, stimiamo invece la probabilità di violazione empirica $\gamma_{n_{\text{int}}}(\theta)$ definita in (3.3), in cui ogni win-rate scenario-specifico è approssimato usando un numero finito n_{int} di partite simulate. Per stimare $\gamma_{n_{\text{int}}}(\theta)$ con garanzie di errore assoluto, il sistema esegue una procedura di Statistical Model Checking (SMC) [1, 2, 9, 15].

La procedura SMC campiona scenari $s^{(1)}, s^{(2)}, \dots$ indipendentemente dalla distribuzione D . Per ciascuno scenario campionato $s^{(k)}$, viene eseguito un piccolo esperimento interno:

1. si gioca un numero fissato n_{int} di partite sotto $s^{(k)}$ usando il controllore finale θ ;
2. si calcola un win-rate empirico $\hat{p}_{n_{\text{int}}}(\theta, s^{(k)})$;
3. si definisce un indicatore di violazione come

$$Z_k = \begin{cases} 1, & \text{se } \hat{p}_{n_{\text{int}}}(\theta, s^{(k)}) < \tau_{\text{verify}}, \\ 0, & \text{altrimenti.} \end{cases}$$

Le variabili aleatorie Z_k sono di Bernoulli con media $\gamma_{n_{\text{int}}}(\theta)$. Dopo N scenari campionati, la probabilità di violazione empirica è

$$\hat{\gamma} = \frac{1}{N} \sum_{k=1}^N Z_k.$$

Un intervallo score di Wilson $[\ell_\gamma, u_\gamma]$ è costruito per $\gamma_{n_{\text{int}}}(\theta)$ al livello di confidenza $1 - \delta$. Il processo di campionamento viene eseguito in modo sequenziale finché

$$\frac{u_\gamma - \ell_\gamma}{2} \leq \varepsilon_\gamma$$

oppure finché non sia stato campionato un numero massimo di scenari. Il criterio di accettazione QG è

$$u_\gamma \leq \varepsilon_{\text{viol}},$$

che supporta, al livello di confidenza $1 - \delta$, l'affermazione che

$$\gamma_{n_{\text{int}}}(\theta) \leq \varepsilon_{\text{viol}}.$$

Lo SMC fornisce quindi garanzie statistiche per $\gamma_{n_{\text{int}}}(\theta)$, che viene usata come proxy della vera probabilità di violazione $\gamma(\theta)$. Il bias di approssimazione si riduce all'aumentare di n_{int} , e con budget interni sufficientemente grandi la quantità empirica si avvicina a quella teorica.

Le garanzie fornite dall'intervallo di confidenza di Wilson sono empiriche e valgono con elevato livello di confidenza in regime di campione finito. Sebbene la regola di arresto sia adattiva, la procedura è intesa come un meccanismo pratico di certificazione ad alta confidenza, piuttosto che come una garanzia formale di inferenza sequenziale.

Se il criterio di accettazione non è soddisfatto, gli scenari campionati con $Z_k = 1$ vengono memorizzati come controesempi e utilizzati per raffinare l'insieme di training.

La logica dell'Algoritmo 6 è implementata in `smc.py`, che definisce il driver SMC usato da `qg_runner.py` durante le fasi di verifica del ciclo QG.

3.7 Corrispondenza tra metodi e moduli Python

Per chiarezza e riproducibilità, riassumiamo brevemente come i principali metodi descritti in questo capitolo sono mappati sulla codebase Python:

- **Simulazione delle partite** (Algoritmo 1) è implementata dal motore scacchistico e dalle funzioni driver in `chess_engine.py`, che espongono routine per inizializzare le posizioni, generare mosse legali, applicare mosse e valutare gli esiti.
- **Stima del win-rate** (Algoritmo 2) è realizzata da funzioni di supporto in `batch_logging.py` e `sequential_verification.py`, che richiamano ripetutamente il simulatore di partite e aggregano gli esiti prodotti.
- **Calcolo della loss di fattibilità** (Algoritmo 3) e l'aggiornamento ES (Algoritmo 4) sono implementati in `es_bbo.py`, nella classe `EvolutionStrategiesBBO` e nelle relative utility.
- **Verifica sequenziale per scenario** (Algoritmo 5) è implementata nel file `sequential_verification.py`. La verifica scenario per scenario è eseguita richiamando ripetutamente il simulatore di partite e aggiornando gli intervalli di Wilson finché la regola di arresto non è soddisfatta.

- **SMC della probabilità di violazione** (Algoritmo 6) è implementato in `smc.py`, che campiona scenari dalla distribuzione configurata in `config.py` e registra i controesempi.
- **Orchestrazione del ciclo QG** (Algoritmo 7) è implementata in `qg_runner.py`, che coordina training ES, verifica per scenario e SMC e aggiorna l'insieme di scenari con i nuovi controesempi trovati.

Questa mappatura esplicita evidenzia che il ciclo QG non è solo specificato a livello astratto, ma è pienamente realizzato in un artefatto software concreto e riproducibile.

3.8 Aspetti non standard di questa istanziazione QG

Il caso di studio scacchistico considerato in questa tesi segue il paradigma generale Quality-Guaranteed, ma combina diverse scelte progettuali che non sono standard né nel reinforcement learning né nella letteratura sui motori di gioco:

- L'ottimizzatore ES è applicato a una *loss di fattibilità* costruita intorno a un requisito di win-rate, invece che a una funzione di reward liscia. Questo sposta l'attenzione dal massimizzare la prestazione media al soddisfare un vincolo esplicito sui win-rate a livello di scenario.
- Lo Statistical Model Checking è utilizzato per stimare la *probabilità di violazione* di questo requisito sotto una distribuzione di scenari, piuttosto che per verificare proprietà in logica temporale di un modello fissato. Il ciclo QG ragiona quindi direttamente in termini di tassi di fallimento del controllore.
- Il raffinamento guidato dai controesempi opera al livello degli *scenari*, aggiungendo seed e politiche avversarie che sono state osservate violare il requisito all'insieme di training. Questo crea un ciclo di feedback in cui la verifica modella attivamente il problema di ottimizzazione, nello spirito di CEGIS [11].
- Il controllore neurale è deliberatamente compatto e adatto alla CPU. L'obiettivo non è competere con i motori allo stato dell'arte in termini di forza di gioco, ma isolare e mettere in evidenza il comportamento del ciclo QG stesso su un problema di controllo realistico ma trattabile.

Nel complesso, queste scelte portano a un'istanza QG che differisce in modo sostanziale dalle pipeline classiche di self-play e dalle tipiche applicazioni di SMC, ed è calibrata sullo scopo di ottenere garanzie esplicite a livello di distribuzione sui win-rate.

3.9 Il ciclo Quality-Guaranteed completo

Mettendo insieme tutti i pezzi, il ciclo QG implementato nel progetto è composto dai seguenti passi.

1. **Inizializzazione.** Si sceglie un insieme iniziale di scenari S_0 , una distribuzione di scenari D , una soglia di training τ_{train} , una soglia di verifica τ_{verify} , parametri di errore assoluto ε e ε_γ , e una tolleranza sulla violazione $\varepsilon_{\text{viol}}$. Si inizializza θ in modo casuale e si pone $S \leftarrow S_0$.
2. **Training ES.** Si esegue l'ottimizzatore ES su $J_S(\theta)$ per un numero fissato di iterazioni o fino alla convergenza, usando il simulatore per valutare la loss.
3. **Verifica per scenario.** Per ogni $s \in S$, si esegue il verificatore sequenziale descritto nella Sezione 3.5, ottenendo $\hat{p}_s$, un intervallo di Wilson e una decisione pass/fail. Si raccoglie l'insieme $F \subseteq S$ degli scenari non superati (cioè gli scenari classificati come FAIL o BORDERLINE).
4. **SMC globale.** Si esegue la procedura SMC della Sezione 3.6 per stimare $\gamma(\theta)$, ottenendo $\hat{\gamma}$ e un intervallo di Wilson $[\ell_\gamma, u_\gamma]$.
5. **Test di arresto.** Se $u_\gamma \leq \varepsilon_{\text{viol}}$, si accetta il controllore: con probabilità almeno $1 - \delta$, la probabilità di violazione è entro la tolleranza. Il ciclo termina.
6. **Raffinamento.** Altrimenti, si raccoglie l'insieme C degli scenari campionati durante lo SMC con $Z_k = 1$ e si aggiorna $S \leftarrow S \cup C$. Facoltativamente, si possono regolare gli iperparametri ES per dare più peso agli scenari in $F \cup C$.
7. **Iterazione.** Si torna al training ES con l'insieme arricchito S .

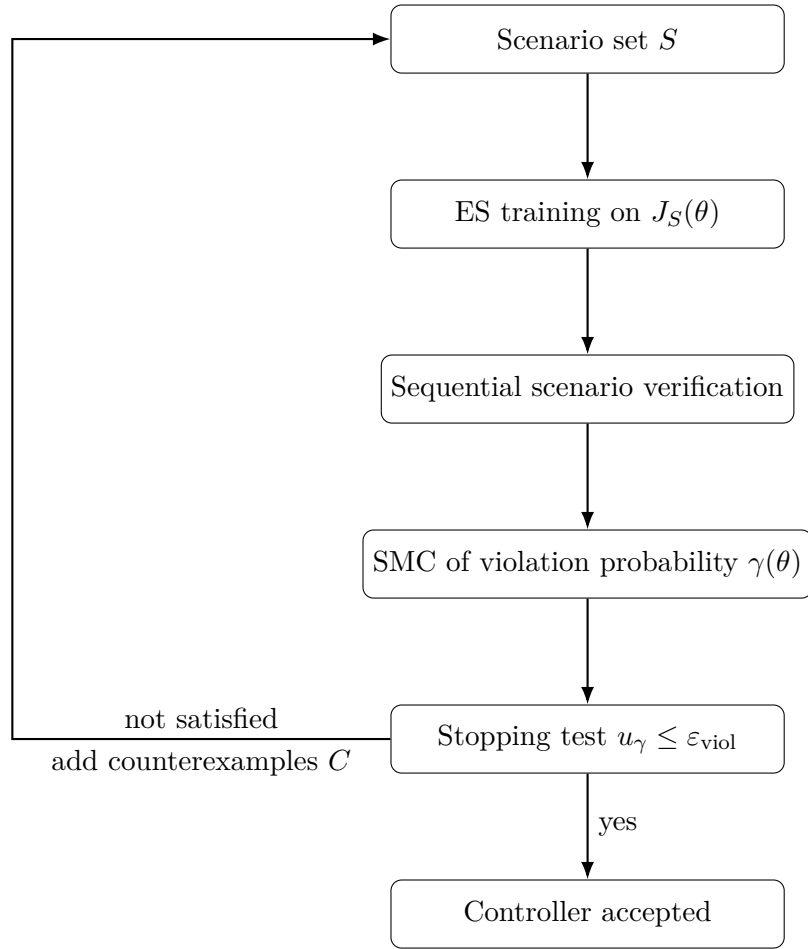


Figura 3.2. Ciclo logico di apprendimento Quality-Guaranteed per gli scacchi.

Sebbene la Figura 4.1 presenti i moduli in una disposizione simile a una pipeline, l'interazione tra i componenti non è strettamente unidirezionale. Sia l'ottimizzatore ES sia i moduli di verifica interrogano ripetutamente il motore e il controllore durante la simulazione delle partite. In questo senso, gli strati più bassi (motore e controllore) agiscono come servizi richiamabili iterativamente dai componenti di livello superiore, piuttosto che come stadi passivi in un unico flusso di dati in avanti.

In questo modo, il ciclo QG implementato nel progetto non tratta apprendimento e verifica come fasi separate. Al contrario, esse formano un unico processo iterativo: ES propone controllori che riducono una loss centrata sul requisito, la verifica valuta la probabilità di violazione con limiti di confidenza espliciti e i controesempi vengono reinseriti nel successivo ciclo di ottimizzazione.

Algorithm 1 SIMULATEMATCH(θ, s)

Require: parametri del controllore θ , scenario $s = (\sigma, o)$ **Ensure:** esito della partita $Y \in \{0, 1\}$

```

1: inizializza lo stato di gioco usando il seed  $\sigma$ 
2: while la partita non è terminale e il numero di plies è  $< N_{\max}$  do
3:   if deve muovere il giocatore progettato then
4:     scegli la mossa con il controllore  $f_\theta$  (opzionalmente  $\varepsilon$ -greedy)
5:   else
6:     scegli la mossa con la politica avversaria  $o$ 
7:   end if
8:   applica la mossa allo stato di gioco
9: end while
10:  $Y \leftarrow 1$  se vittoria, altrimenti 0 ▷ patta/sconfitta contano come fallimento
11: return  $Y$ 

```

Algorithm 2 EstimateWinRate(θ, s, n)

Require: parametri del controllore θ , scenario $s = (\sigma, o)$, numero di partite n **Ensure:** win-rate empirico $\hat{p}(\theta, s)$

```

1:  $k \leftarrow 0$  ▷ contatore di vittorie
2: for  $i \leftarrow 1$  to  $n$  do
3:    $\sigma_i \leftarrow \text{DERIVESEED}(\sigma, i)$ 
4:    $Y_i \leftarrow \text{SIMULATEMATCH}(\theta, (\sigma_i, o))$ 
5:    $k \leftarrow k + Y_i$ 
6: end for
7:  $\hat{p}(\theta, s) \leftarrow k/n$ 
8: return  $\hat{p}(\theta, s)$ 

```

Algorithm 3 COMPUTEF easibilityLOSS($\theta, S, \tau_{\text{train}}, n_{\text{games}}$)

Require: parametri del controllore θ , insieme di scenari di training S , soglia di win-rate τ_{train} , numero di partite per scenario n_{games} **Ensure:** loss di fattibilità $J_S(\theta)$

```

1:  $J_S(\theta) \leftarrow 0$ 
2: for all  $s \in S$  do
3:    $\hat{p}(\theta, s) \leftarrow \text{ESTIMATEWINRATE}(\theta, s, n_{\text{games}})$ 
4:    $d_s \leftarrow \max(0, \tau_{\text{train}} - \hat{p}(\theta, s))$ 
5:    $J_S(\theta) \leftarrow J_S(\theta) + d_s$ 
6: end for
7: return  $J_S(\theta)$ 

```

Algorithm 4 Addestramento tramite Evolution Strategies sulla loss di fattibilità $J_S(\theta)$

Require: parametri iniziali θ , insieme di scenari S , dimensione della popolazione λ , scala delle perturbazioni σ , learning rate α , numero di partite per scenario n_{games}

Ensure: parametri aggiornati θ

```

1: for iterazione = 1, 2, ... finché non è soddisfatto il criterio di arresto do
2:   for  $i = 1$  to  $\lambda$  do
3:     campiona  $\varepsilon_i \sim \mathcal{N}(0, \sigma^2 I)$ 
4:      $\theta_i \leftarrow \theta + \varepsilon_i$ 
5:     for all  $s \in S$  do
6:        $\hat{p}(\theta_i, s) \leftarrow \text{ESTIMATEWINRATE}(\theta_i, s, n_{\text{games}})$ 
7:     end for
8:     calcola la loss di fattibilità

```

$$J_S(\theta_i) = \sum_{s \in S} \max(0, \tau_{\text{train}} - \hat{p}(\theta_i, s))$$

```

9:   end for
10:  ordina i candidati  $\{\theta_i\}$  per loss crescente  $J_S(\theta_i)$ 
11:  assegna pesi normalizzati  $\{w_i\}_{i=1}^\lambda$ 
12:  aggiorna i parametri

```

$$\theta \leftarrow \theta + \alpha \sum_{i=1}^{\lambda} w_i \varepsilon_i$$

```

13:  if il criterio di convergenza è soddisfatto o il numero massimo di iterazioni è
    raggiunto then
14:    break
15:  end if
16: end for
17: return  $\theta$ 

```

Algorithm 5 Verifica sequenziale scenario per scenario mediante intervalli di Wilson

Require: controllore θ , insieme di scenari S , soglia τ_{verify} , accuratezza ε , livello di confidenza $1 - \delta$, numero massimo di partite per scenario n_{max}

Ensure: stime $\{\hat{p}(\theta, s)\}_{s \in S}$ e insieme degli scenari non superati F

```

1:  $F \leftarrow \emptyset$ 
2: for all  $s \in S$  do
3:    $k_s \leftarrow 0$  ▷ vittorie
4:    $n_s \leftarrow 0$  ▷ partite
5:    $\text{status}(s) \leftarrow \text{UNDECIDED}$ 
6: end for
7: while esiste uno scenario  $s$  con  $\text{status}(s) = \text{UNDECIDED}$  do
8:   for all  $s$  con  $\text{status}(s) = \text{UNDECIDED}$  do
9:     gioca un batch di partite sotto  $(\theta, s)$ 
10:    aggiorna  $k_s$  e  $n_s$ 
11:     $\hat{p}(\theta, s) \leftarrow k_s/n_s$ 
12:    calcola l'intervallo di Wilson  $[\ell_s, u_s]$  al livello  $1 - \delta$ 
13:     $r_s \leftarrow (u_s - \ell_s)/2$ 
14:    if  $r_s \leq \varepsilon$  oppure  $n_s \geq n_{\text{max}}$  then
15:      if  $\ell_s \geq \tau_{\text{verify}}$  then
16:         $\text{status}(s) \leftarrow \text{PASS}$ 
17:      else if  $u_s < \tau_{\text{verify}}$  then
18:         $\text{status}(s) \leftarrow \text{FAIL}$ 
19:         $F \leftarrow F \cup \{s\}$ 
20:      else
21:         $\text{status}(s) \leftarrow \text{BORDERLINE}$ 
22:         $F \leftarrow F \cup \{s\}$ 
23:      end if
24:    end if
25:  end for
26: end while
27: return  $\{\hat{p}(\theta, s)\}_{s \in S}, F$ 

```

Algorithm 6 SMCVIOLATION($\theta, D, \tau_{\text{verify}}, n_{\text{int}}, \varepsilon_\gamma, \delta, N_{\text{max}}$)

Require: parametri del controllore θ , distribuzione di scenari D , soglia di verifica τ_{verify} , numero di partite interne per scenario n_{int} , accuratezza ε_γ , livello di confidenza $1 - \delta$, numero massimo di scenari campionati N_{max}

Ensure: stima empirica della violazione $\hat{\gamma}$, intervallo di Wilson $[\ell_\gamma, u_\gamma]$ per $\gamma_{n_{\text{int}}}(\theta)$, e insieme di controesempi C

```

1:  $C \leftarrow \emptyset$ 
2:  $N \leftarrow 0$  ▷ numero di scenari campionati
3:  $K \leftarrow 0$  ▷ numero di violazioni
4: repeat
5:   campiona uno scenario  $s \sim D$ 
6:    $\hat{p}_{n_{\text{int}}}(\theta, s) \leftarrow \text{ESTIMATEWINRATE}(\theta, s, n_{\text{int}})$ 
7:   if  $\hat{p}_{n_{\text{int}}}(\theta, s) < \tau_{\text{verify}}$  then
8:      $K \leftarrow K + 1$ 
9:      $C \leftarrow C \cup \{s\}$ 
10:  end if
11:   $N \leftarrow N + 1$ 
12:   $\hat{\gamma} \leftarrow K/N$ 
13:  calcola l'intervallo di Wilson  $[\ell_\gamma, u_\gamma]$  per la probabilità Bernoulli di violazione
    sulla base di  $(K, N)$ 
14:   $r_\gamma \leftarrow (u_\gamma - \ell_\gamma)/2$ 
15: until  $r_\gamma \leq \varepsilon_\gamma$  oppure  $N \geq N_{\text{max}}$ 
16: return  $(\hat{\gamma}, [\ell_\gamma, u_\gamma], C)$ 

```

Algorithm 7 Ciclo di apprendimento Quality-Guaranteed (QG) per gli scacchi

Require: insieme iniziale di scenari S_0 , distribuzione di scenari D , soglia di training τ_{train} , soglia di verifica τ_{verify} , numero di partite per scenario n_{games} , numero di partite interne per scenario n_{int} , accuratezza locale ε , accuratezza globale ε_γ , livello di confidenza $1 - \delta$, numero massimo di scenari campionati N_{max} , tolleranza sulla violazione $\varepsilon_{\text{viol}}$

Ensure: parametri finali del controllore θ

```

1:  $S \leftarrow S_0$ 
2: inizializza  $\theta$  in modo casuale
3: while true do
4:    $\theta \leftarrow \text{ESTRAIN}(\theta, S, \tau_{\text{train}}, n_{\text{games}})$ 
5:    $(\{\hat{p}_s\}_{s \in S}, F) \leftarrow \text{SEQUENTIALVERIFY}(\theta, S, \tau_{\text{verify}}, \varepsilon, \delta, n_{\text{max}})$ 
6:    $(\hat{\gamma}, [\ell_\gamma, u_\gamma], C) \leftarrow \text{SMCVIOLATION}(\theta, D, \tau_{\text{verify}}, n_{\text{int}}, \varepsilon_\gamma, \delta, N_{\text{max}})$ 
7:   if  $u_\gamma \leq \varepsilon_{\text{viol}}$  then
8:     return  $\theta$ 
9:   else
10:     $S \leftarrow S \cup C$ 
11:  end if
12: end while

```

Capitolo 4

Implementazione

Questo capitolo spiega come i metodi del Capitolo 3 siano realizzati nel progetto Python effettivo. L'obiettivo non è ripetere la teoria, bensì mostrare in modo concreto e riproducibile come gli oggetti astratti del framework Quality-Guaranteed (QG) — scenari, win-rate, loss di fattibilità, Evolution Strategies (ES), intervalli di Wilson e Statistical Model Checking (SMC) — vengano trasformati in moduli, funzioni e strutture dati, seguendo i riferimenti classici su ES [6, 7, 8], intervalli di Wilson [14], e SMC [1, 2, 9, 15].

Tutti i componenti descritti in questo capitolo fanno parte della codebase pubblica del progetto. Gli elementi centrali del ciclo QG sono implementati nei seguenti file:

- `chess_engine.py`: motore scacchistico e simulazione delle partite;
- `nn_player.py` e `nn_core.py`: controllore e rete neurale;
- `es_bbo.py`: ottimizzatore ES;
- `sequential_verification.py`: verifica scenario per scenario;
- `smc.py`: SMC globale;
- `qg_runner.py`: driver QG e orchestrazione.

L'implementazione segue da vicino gli algoritmi del Capitolo 3.

4.1 Panoramica dell'architettura software

L'implementazione è organizzata come uno stack leggero di moduli e package Python, ognuno dei quali rispecchia uno strato logico della pipeline QG introdotta nel Capitolo 3. Una vista schematica è mostrata nella Figura 4.1.

Nell'implementazione di riferimento questi strati sono realizzati come package Python (per esempio, `engine`, `controller`, `optim`, `verify`, `scenarios`, `utils`). La struttura precisa delle directory è convenzionale e può essere modificata senza toccare la logica sottostante, purché le stesse interfacce siano preservate.

A runtime, il flusso dei dati può essere riassunto come segue:

1. lo scenario manager costruisce o seleziona uno scenario s e lo passa allo strato ES (durante l'ottimizzazione) oppure allo strato di verifica (durante i controlli);
2. il motore inizializza la corrispondente posizione di scacchi, restituendo un oggetto `GameState` che rappresenta la scacchiera corrente;
3. il controllore codifica lo stato in un vettore, invoca la rete neurale per ottenere uno score e sceglie una mossa secondo una politica ε -greedy;
4. il motore applica la mossa, interroga la politica avversaria e fa avanzare lo stato fino a una condizione terminale oppure al raggiungimento di un limite massimo di plies;
5. gli esiti delle partite vengono propagati verso l'alto e aggregati in win-rate, loss di fattibilità e indicatori di violazione dai moduli ES e di verifica.

Ogni passaggio istanzia, in corrispondenza biunivoca, il corrispondente oggetto matematico del Capitolo 3: una singola partita realizza il trial di Bernoulli SIMULATMATCH dell'Algoritmo 1, i win-rate empirici sono calcolati come nell'Algoritmo 2, il modulo ES minimizza $J_S(\theta)$ in Equazione (3.4), e i moduli di verifica implementano le procedure sequenziale e SMC degli Algoritmi 5 e 6.

4.2 Scelte ingegneristiche

L'architettura è stata progettata intenzionalmente per essere semplice, trasparente e riproducibile, più che competitiva con i motori scacchistici di produzione. Diverse decisioni progettuali sono state guidate da questo vincolo e dalla necessità di mantenere il ciclo QG eseguibile su un normale portatile.

Ambiente di esecuzione. Tutti gli esperimenti sono eseguiti su una CPU multi-core standard, usando Python 3 e soltanto librerie numeriche leggere. L'implementazione non richiede GPU. Questo rispecchia l'obiettivo progettuale del Capitolo 3: il principale collo di bottiglia è il numero di partite simulate richieste da ES e SMC, non la complessità delle singole forward pass.

Motore personalizzato invece di librerie esterne. Invece di delegare la logica di gioco a un package esistente come `python-chess` [4], il progetto utilizza un motore compatto sviluppato internamente e incorporato in `chess_engine.py`. Questo elimina una dipendenza esterna e garantisce che l'esatta funzione di transizione usata negli esperimenti sia completamente visibile e modificabile. Inoltre rimuove ogni ambiguità su come vengano gestite le regole speciali (arrocco, *en passant*, ripetizioni): il comportamento usato nell'analisi QG coincide esattamente con quello implementato nel simulatore.

Controllore solo-CPU. Il controllore neurale è deliberatamente poco profondo e compatto, con un'architettura completamente connessa implementata in `nn_core.py`. Il controllore può essere valutato milioni di volte in una singola esecuzione senza richiedere hardware specializzato. Dal punto di vista QG, questo è preferibile a un

modello più profondo: il collo di bottiglia è il numero di partite richieste da ES e SMC, non il potere espressivo della rete.

ES invece di RL basato su gradiente. Il reinforcement learning basato su gradiente richiederebbe un ciclo di addestramento più complesso, target espliciti di valore o politica e diversi accorgimenti di stabilizzazione. In questo progetto la quantità obiettivo è la loss di fattibilità non differenziabile $J_S(\theta)$, costruita a partire da win-rate empirici e operazioni di parte positiva come in Equazione (3.4). ES, implementato in `es_bbo.py`, tratta l'intero simulatore — motore, avversari e controllore — come una black box e richiede soltanto loss scalari. Lo stesso ottimizzatore funzionerebbe senza modifiche se il motore o il controllore fossero sostituiti, in linea con le formulazioni standard di ES [6, 7, 8].

Progettazione centrata sugli scenari. Infine, la scelta di esporre gli scenari come oggetti di prima classe riflette direttamente il paradigma QG formalizzato nella Sezione 3.2. Lo scenario manager tiene traccia dell'insieme corrente S , dell'insieme iniziale S_0 e del buffer di controesempi C . ES non si allena mai su un generico “flusso di partite”; opera sempre su un insieme finito esplicito di scenari che cresce nel tempo, rendendo l'evoluzione del ciclo QG più facile da ispezionare e debuggare.

4.3 Motore scacchistico

Il motore scacchistico è responsabile della rappresentazione degli stati di gioco e del loro avanzamento secondo le regole degli scacchi. Nell'implementazione, una posizione è rappresentata da un'istanza della classe `GameState` in `chess_engine.py`. La classe mantiene:

- l'occupazione di ciascuna delle 8×8 caselle (tipo di pezzo o vuota);
- il lato cui spetta la mossa;
- i diritti di arrocco per entrambi i colori;
- un'eventuale colonna target per *en passant*;
- i contatori di mezzamossa e mossa piena.

Questa rappresentazione coincide con la nozione astratta di posizione in Sezione 2.2. Tramite l'interfaccia pubblica di `GameState`, il motore espone le due operazioni di base usate in tutto il progetto:

- `generate_legal_moves()`, che restituisce tutte le mosse legali nello stato corrente;
- `make_move(move)`, che applica una mossa e restituisce lo stato successore.

La logica interna segue la strategia in due fasi descritta nella Sezione 2.2. Per ogni pezzo appartenente al lato di mossa, il motore costruisce innanzitutto una lista di mosse *pseudo-legali*: mosse che rispettano le regole di movimento del pezzo ma

ignorano temporaneamente la sicurezza del re. Ogni mossa candidata viene poi testata applicandola su una copia temporanea della posizione e verificando se il re del giocatore di mossa resti sotto scacco. Le mosse che falliscono questo test vengono scartate.

Le mosse speciali sono gestite esplicitamente:

- l'*arrocco* è consentito solo se né il re né la torre si sono già mossi, le caselle intermedie sono vuote e nessuna delle caselle attraversate dal re è sotto attacco;
- le catture *en passant* sono permesse solo nella mossa immediatamente successiva a una spinta di due caselle di un pedone avversario, e solo dalle caselle di pedoni adiacenti ammissibili;
- le mosse di *promozione* portano il tipo di pezzo di promozione; il motore genera automaticamente una mossa legale per ogni pezzo di promozione consentito.

Le condizioni di patta (stallo, posizioni morte dovute a materiale insufficiente, ripetizioni e regola delle cinquanta mosse) sono verificate alla fine di ogni mossa. Il motore mantiene le informazioni necessarie affinché le partite terminino in tempo finito anche in posizioni patologiche. Inoltre, il ciclo QG impone un limite globale al numero di plies per partita; se tale limite viene raggiunto, la partita è considerata non vincente per il controllore, in accordo con il vincolo di tipo must-win usato per definire gli esiti Bernoulli nella Sezione 2.5.

Una piccola suite di unit test accompagna `GameState`. Questi test coprono il movimento di base di ciascun pezzo, il rilevamento di mosse illegali che lasciano il re sotto scacco, le regole di arrocco, i casi limite di *en passant* e le patte per ripetizione. Essi fungono da suite di regressione e forniscono fiducia pratica nel fatto che il simulatore usato da ES e SMC rispetti le regole assunte nei Capitoli 2 e 3.

4.4 Codifica dello stato e rete neurale

Lo strato del controllore collega lo stato simbolico di gioco mantenuto dal motore con il vettore numerico atteso dalla rete neurale. Nell'implementazione, questa mappatura è fornita da funzioni di utilità in `nn_player.py` e `nn_core.py`, concretizzando la pipeline di codifica della Sezione 2.3.

Dato un oggetto `GameState`, l'encoder costruisce un vettore $x \in \mathbb{R}^d$ secondo il seguente schema:

1. **Mappe binarie dei pezzi.** Per ogni tipo di pezzo e colore viene creato un piano binario 8×8 . L'elemento associato a una casella vale 1 se la casella contiene quel tipo di pezzo, e 0 altrimenti.
2. **Feature globali.** Un piccolo insieme di feature scalari codifica il lato di mossa, i diritti di arrocco e informazioni di base sul conteggio delle mosse.
3. **Flattening.** Tutti i piani e le feature scalari vengono concatenati e appiattiti in un vettore x di dimensione fissa `D_INPUT`, definita in `nn_player.py`.

L'encoder è deterministico: lo stesso `GameState` produce sempre lo stesso vettore. Questa proprietà è cruciale quando gli scenari vengono riprodotti con lo stesso seed durante il debug o la verifica.

La rete neurale vera e propria è un modello feed-forward completamente connesso $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}$, implementato in `nn_core.py`. Il modello consiste in uno strato di input di dimensione `D_INPUT`, uno o più strati nascosti con funzioni di attivazione non lineari e un'uscita scalare finale. Tutti i pesi e i bias sono memorizzati in un singolo vettore di parametri θ , che è l'oggetto manipolato da ES. L'accesso a θ è incapsulato in una sottile classe wrapper `NNPlayer` in `nn_player.py`.

La selezione delle mosse funziona nel modo seguente, concretizzando la politica descritta nella Sezione 2.3 e nell'Algoritmo 1:

1. il controllore invoca `generate_legal_moves()` sullo `GameState` corrente;
2. ogni mossa è applicata virtualmente per ottenere uno stato successore;
3. lo stato successore è codificato in un vettore x e valutato dalla rete come $f_\theta(x)$;
4. viene selezionata la mossa con il punteggio più alto, eventualmente sostituita con una mossa casuale con probabilità ε (un meccanismo di esplorazione ε -greedy [12]).

Questa procedura concretizza il controllore astratto discusso nel Capitolo 3 in un oggetto Python che può essere inserito nel simulatore o in una GUI interattiva.

4.5 Modulo Evolution Strategies

L'ottimizzatore ES è implementato in `es_bbo.py` e si basa sulla loss di fattibilità $J_S(\theta)$ definita in Equazione (3.4). Il modulo espone una classe `EvolutionStrategiesBBO` che mantiene:

- il vettore di parametri corrente θ ;
- gli iperparametri ES: dimensione di popolazione λ , ampiezza delle perturbazioni σ , learning rate α ;
- l'insieme di scenari di training corrente S ;
- la configurazione per il numero di partite per scenario e per individuo.

Ogni iterazione ES segue i passi dell'Algoritmo 4:

1. si campionano perturbazioni ε_i e si costruiscono i candidati $\theta_i = \theta + \varepsilon_i$;
2. per ogni θ_i , il simulatore viene invocato per stimare i win-rate empirici $\hat{p}_s(\theta_i)$ su tutti gli scenari $s \in S$;
3. si calcola la loss di fattibilità $J_S(\theta_i)$ a partire da tali win-rate;
4. si ordinano i candidati per loss e si convertono i ranghi in pesi normalizzati w_i ;

5. si aggiorna θ secondo la regola ES in Equazione (3.5).

Le valutazioni delle partite sono parallelizzate tramite un pool di processi worker. A ciascun worker viene assegnato un blocco di parametri candidati e scenari da valutare, che vengono simulati richiamando ripetutamente le routine di simulazione delle partite in `chess_engine.py`. Le statistiche aggregate (vittorie, sconfitte, numero di partite) sono poi restituite al processo principale, che calcola la loss di fattibilità e aggiorna θ .

Il modulo ES registra, per ogni iterazione, il valore di $J_S(\theta)$, il win-rate medio sull'insieme di training corrente S e informazioni temporali di base. Questi log vengono poi riutilizzati nel capitolo sperimentale per tracciare le curve di apprendimento e diagnosticare eventuali problemi di convergenza.

4.6 Modulo di verifica sequenziale

La verifica scenario per scenario è implementata in `sequential_verification.py`. Il modulo fornisce una funzione che prende in input i parametri finali del controllore θ , l'insieme di scenari S , la soglia τ_{verify} , i parametri di accuratezza e confidenza, e restituisce un report dettagliato per ogni scenario.

Internamente, per ciascun $s \in S$, il verificatore mantiene:

- i contatori di vittorie e partite (k_s, n_s) ;
- il win-rate empirico corrente $\hat{p}_s$;
- l'intervallo di Wilson corrente $[\ell_s, u_s]$ [14];
- un flag di stato (undecided, pass, fail o borderline).

La verifica procede a round, seguendo l'Algoritmo 5:

1. si individuano tutti gli scenari il cui stato è ancora undecided;
2. per ciascuno di questi scenari si gioca un nuovo batch di partite sotto il controllore θ , aggiornando k_s e n_s ;
3. si ricalcolano $\hat{p}_s$ e l'intervallo di Wilson $[\ell_s, u_s]$;
4. se il raggio dell'intervallo è al di sotto dell'accuratezza target ϵ , oppure se è stato esaurito il budget massimo di partite, lo scenario viene classificato come pass, fail o borderline secondo le regole della Sezione 3.5.

Il calcolo dell'intervallo di Wilson [14] è incapsulato in una piccola funzione di utilità, riutilizzata dal motore SMC per evitare duplicazioni di codice. Al termine dell'esecuzione, il verificatore scrive un riepilogo machine-readable (per esempio, un file CSV) contenente una riga per scenario con il win-rate finale, l'intervallo di confidenza, lo stato e il numero di partite. Questo file funge sia da documentazione della fase di verifica sia da input per la gestione dei controesempi.

4.7 Motore di Statistical Model Checking

Il componente SMC è implementato in `smc.py`. Il suo ruolo è stimare la probabilità di violazione empirica $\gamma_{n_{\text{int}}}(\theta)$, che funge da approssimazione Monte Carlo della vera probabilità di violazione $\gamma(\theta)$ definita in Equazione (3.2), insieme a una garanzia di errore assoluto, come formalizzato nella Sezione 3.6 e nella letteratura su SMC [1, 2, 9, 15].

Il motore SMC prende come argomenti:

- la distribuzione di scenari D , implementata come campionatore congiunto su seed e identificativi di avversario;
- i parametri finali del controllore θ ;
- il numero di partite interne per scenario campionato;
- i parametri di accuratezza e confidenza ε_γ e δ .

Il motore esegue quindi l'Algoritmo 6: campiona ripetutamente scenari $s^{(k)} \sim D$, gioca il numero prescritto di partite sotto ciascuno di essi, stima un win-rate a livello di scenario e lo converte in un indicatore di violazione Z_k . Dopo ogni nuovo scenario, aggiorna la probabilità di violazione empirica $\hat{\gamma}$ e l'intervallo di Wilson $[\ell_\gamma, u_\gamma]$. Il processo si arresta quando il raggio dell'intervallo scende sotto ε_γ oppure quando viene raggiunto un numero massimo prefissato di scenari campionati.

Tutti gli scenari con $Z_k = 1$ vengono salvati in una lista di controesempi insieme al win-rate stimato. La funzione restituisce la stima finale $\hat{\gamma}$, il corrispondente intervallo di Wilson, il numero di scenari campionati e la lista di controesempi. Queste uscite sono utilizzate da `qg_runner.py` per decidere se arrestare il ciclo QG oppure arricchire l'insieme di training.

4.8 Gestione di scenari e controesempi

Gli scenari sono rappresentati come oggetti Python leggeri che contengono:

- un seed intero a 64 bit σ ;
- un identificativo dell'avversario (per esempio, *random* oppure *greedy-material-with-noise*);
- eventuali metadati, quali l'ultimo win-rate stimato o un tag che marca lo scenario come controesempio.

Questa rappresentazione concretizza la nozione astratta di scenario $s = (\sigma, o)$ introdotta nella Sezione 3.2. Lo scenario manager, implementato come piccolo modulo nel package `scenarios`, fornisce utilità per:

- costruire un insieme iniziale di training S_0 combinando una lista fissata di avversari con seed campionati da un certo intervallo;

- serializzare e deserializzare insiemi di scenari da e verso disco (per esempio in formato JSON);
- fondere una o più liste di controesempi nell'insieme corrente S , evitando duplicati;
- opzionalmente prioritizzare gli scenari in S assegnando loro pesi basati sulle prestazioni passate durante il training ES.

Alla fine di ogni iterazione QG, `qg_runner.py` raccoglie gli scenari falliti dal verificatore sequenziale e quelli violati da SMC, li unisce in un buffer C e aggiorna l'insieme di training a $S \leftarrow S \cup C$, come descritto nella Sezione 1 e negli approcci di sintesi guidata da controesempi [11]. Le successive esecuzioni ES operano così su un insieme più ricco di situazioni, e la loss di fattibilità diventa progressivamente più severa.

4.9 Logging, riproducibilità e strumenti

La riproducibilità è una parte centrale del progetto: una volta eseguito un run QG, dovrebbe essere possibile rieseguirlo ottenendo la stessa sequenza di partite e le stesse statistiche. A questo scopo, l'implementazione include uno strato di logging e configurazione semplice ma esplicito.

A livello di singole partite, le utilità di logging (implementate nei seguenti file `match_logging.py` e `batch_logging.py`) registrano:

- l'identificativo di scenario e il seed;
- una rappresentazione compatta della sequenza di mosse;
- l'esito finale e la lunghezza della partita.

A livello di run completi (ES, verifica sequenziale e SMC), il sistema registra:

- la configurazione completa utilizzata (iperparametri ES, soglie QG, insiemi di scenari);
- le traiettorie della loss e i win-rate aggregati rispetto al numero di iterazione;
- le stime SMC di $\gamma(\theta)$ e i relativi intervalli di confidenza;
- informazioni temporali di alto livello per ciascuna fase del ciclo QG.

I parametri di configurazione sono centralizzati nel modulo `config.py`, che viene importato da tutti gli altri moduli. I seed pseudo-casuali vengono estratti da questa configurazione e propagati al motore, al controllore, al codice ES e di verifica, in modo che il comportamento pseudo-casuale dell'intero sistema possa essere controllato da un unico punto. In pratica, ciò implica che rieseguendo `qg_runner.py` con lo stesso file di configurazione si ottenga la stessa sequenza di partite, salvo differenze numeriche dovute all'aritmetica in virgola mobile.

Oltre ai componenti principali, è disponibile una leggera interfaccia grafica, costruita sullo stesso motore e controllore, che consente di ispezionare le partite registrate e di giocare interattivamente contro il controllore addestrato. Pur non essendo essenziale per la metodologia QG in sé, questa interfaccia è utile come controllo di coerenza: fornisce una vista leggibile del comportamento appreso e aiuta a verificare che le garanzie teoriche siano effettivamente associate a un controllore che gioca partite legali e riconoscibilmente “scacchistiche”.

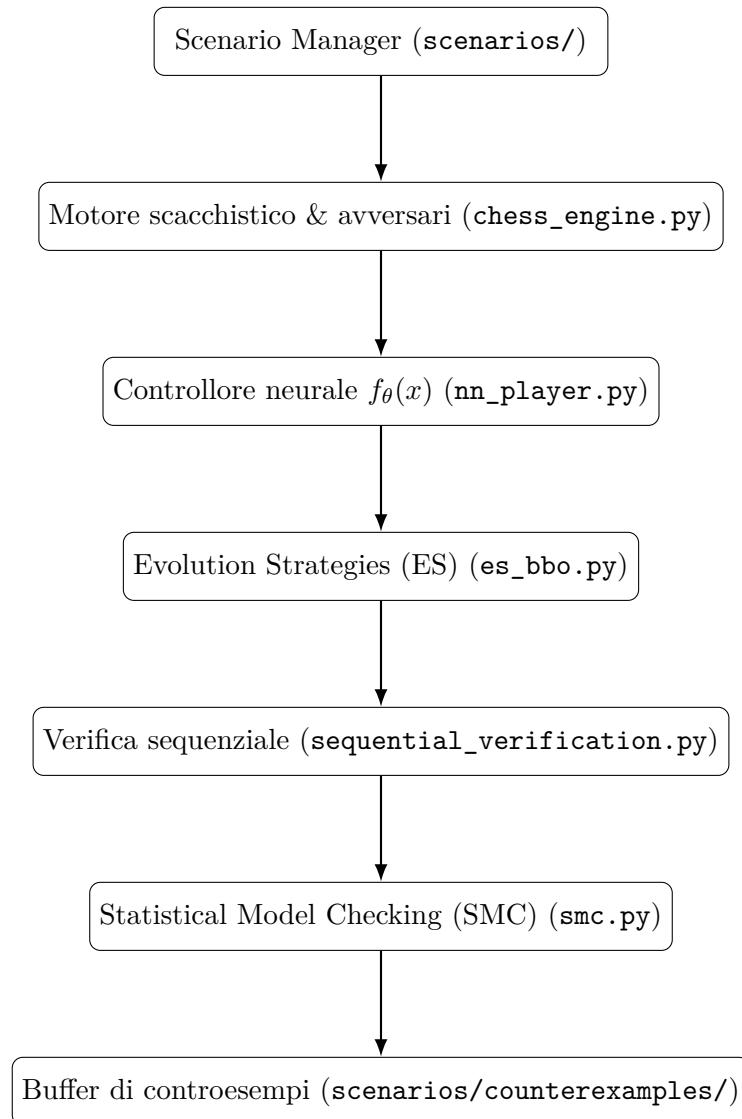


Figura 4.1. Architettura software del sistema scacchistico Quality-Guaranteed. I rettangoli indicano moduli o package Python; le frecce indicano la principale direzione del flusso di dati.

Capitolo 5

Risultati sperimentali

In questo capitolo viene analizzato il comportamento empirico del caso di studio sugli scacchi sotto il ciclo di apprendimento Quality-Guaranteed (QG) descritto nel Capitolo 3 e implementato nel Capitolo 4. Gli obiettivi sono:

- documentare come, in pratica, Evolution Strategies (ES), verifica sequenziale e Statistical Model Checking (SMC) interagiscono nel dominio degli scacchi;
- quantificare, con espliciti intervalli di confidenza, i valori di tasso di vittoria e di probabilità di violazione del controllore neurale rispetto al requisito della Sezione 3.2;
- rendicontare il budget complessivo di simulazioni utilizzato da ciascun componente del ciclo QG.

Invece di concentrarsi su singoli dettagli implementativi, questo capitolo tratta l'implementazione come un'unica pipeline sperimentale completa e analizza il comportamento di training sotto ES, i risultati della verifica scenario per scenario, le stime SMC della probabilità di violazione, le statistiche a livello di partita contro un insieme di avversari di riferimento e la distribuzione dei costi di simulazione tra run e fasi.

5.1 Setup sperimentale

Tutti gli esperimenti seguono il framework formale del Capitolo 3. Qui si riassumono brevemente le scelte concrete rilevanti per l'interpretazione di grafici e tabelle.

5.1.1 Controllore e avversari

In tutti i run, il controllore è la rete neurale feed-forward $f_{\theta}(x)$ introdotta nella Sezione 4.4, incapsulata nella classe `NNPlayer`. L'architettura della rete e la codifica dell'input rimangono fisse per tutta la durata degli esperimenti; solo il vettore di parametri θ viene ottimizzato.

La libreria di avversari rispecchia le politiche utilizzate nella definizione degli scenari in Sezione 3.2. In particolare, gli esperimenti utilizzano:

- un avversario *random*, che seleziona a ogni posizione una mossa legale uniformemente a caso;
- un avversario *greedy-material*, che valuta le mosse tramite una euristica di materiale e sceglie la mossa che massimizza il guadagno materiale immediato;
- un avversario *PSQT-greedy*, che combina una valutazione tramite piece-square table con una regola di scelta greedy;
- un avversario *minimax* con profondità limitata, che esplora un piccolo albero di continuazioni con una ricerca minimax a profondità fissata e una funzione di valutazione semplice [5].

Questi avversari non sono pensati per approssimare i motori moderni; sono baseline semplici e trasparenti che mettono in evidenza aspetti diversi del comportamento del controllore.

5.1.2 Scenari e requisiti

Gli scenari sono definiti esattamente come nella Sezione 3.2: ogni scenario $s = (\sigma, o)$ fissa un seed pseudo-casuale σ e una politica avversaria o . Il seed inizializza tutti i generatori pseudo-casuali (esplorazione del controllore, rumore dell'avversario e tie-breaking nella selezione delle mosse), rendendo ciascun scenario completamente riproducibile.

Gli esperimenti distinguono tra:

- un insieme finito di *training* S , usato in ES e nella verifica sequenziale scenario per scenario;
- una distribuzione di scenari D , che campiona seed e tipi di avversario e viene utilizzata da SMC per stimare la probabilità di violazione $\gamma(\theta)$ in (3.2).

Per distinguere tra la guida all'ottimizzazione e le garanzie di prestazione finali, in questo studio vengono impiegate due diverse soglie di successo. Durante il training ES, la perdita di fattibilità è definita rispetto a una soglia $\tau = 0.5$, che impone un vincolo di vittoria obbligata a livello dei singoli scenari in S . La verifica sequenziale scenario per scenario ricalcola poi i tassi di vittoria per scenario a una soglia più severa $\tau = 0.6$, mentre SMC valuta la probabilità di violazione globale $\gamma(\theta)$ sotto D . Come nel resto della tesi, le patte sono trattate come fallimenti, quindi il requisito è a tutti gli effetti un vincolo di *must-win*. Nel framework astratto del Capitolo 3, soglie tipiche si collocano nell'intervallo 0.7–0.9; per questo caso di studio concreto, le scelte più moderate $\tau = 0.5$ e $\tau = 0.6$ rappresentano un compromesso pragmatico tra un requisito non banale e un budget di simulazione sostenibile.

5.1.3 Budget di simulazione e criteri di arresto

Per ogni combinazione di parametri del controllore, scenario e tipo di esperimento, può essere simulato solo un numero finito di partite. I risultati numerici riportati di seguito sono quindi sempre basati su *stime*:

- durante il training ES, la perdita di fattibilità $J_S(\theta)$ viene valutata usando un numero fisso di partite per scenario, scambiando accuratezza con velocità;
- durante la verifica scenario per scenario, il numero di partite per scenario è scelto in modo adattivo dalla procedura sequenziale basata su intervalli di Wilson della Sezione 3.5, fino a un tetto massimo (50 partite nei run QualityA/B e 75 partite nei run QualityC/D);
- durante SMC, il numero di scenari campionati da D cresce sequenzialmente finché l'intervallo di Wilson per $\gamma(\theta)$ non raggiunge l'accuratezza target o non viene raggiunto un numero massimo di scenari.

In tutti i casi, le regole di arresto sono guidate dalle garanzie di errore assoluto introdotte nel Capitolo 3: vengono simulate partite o scenari aggiuntivi solo finché contribuiscono a ridurre in modo significativo l'incertezza statistica.

Gli esperimenti riportati in questo capitolo si basano su quattro run QG (`logs_runQualityA-D`), più un run più ricco con solo ES (`logs_runRich2`) utilizzato per stress-testare il componente di ottimizzazione. Il budget di simulazione risultante è riassunto in Tabella 5.3 nella Sezione 5.5.

5.2 Comportamento del training ES

Si analizza innanzitutto il comportamento dell'ottimizzatore ES durante la minimizzazione della perdita di fattibilità $J_S(\theta)$ in (3.4). Le grandezze principali di interesse sono:

- i tassi di vittoria empirici per scenario ottenuti al termine di ES;
- la variabilità delle prestazioni di training tra seed e tra run;
- statistiche aggregate come il tasso di vittoria mediano e quello peggiore.

La Figura 5.1 visualizza il tasso di vittoria finale in ES per ciascun scenario e run di qualità, mentre la Figura 5.2 aggrega le distribuzioni tra i vari run.

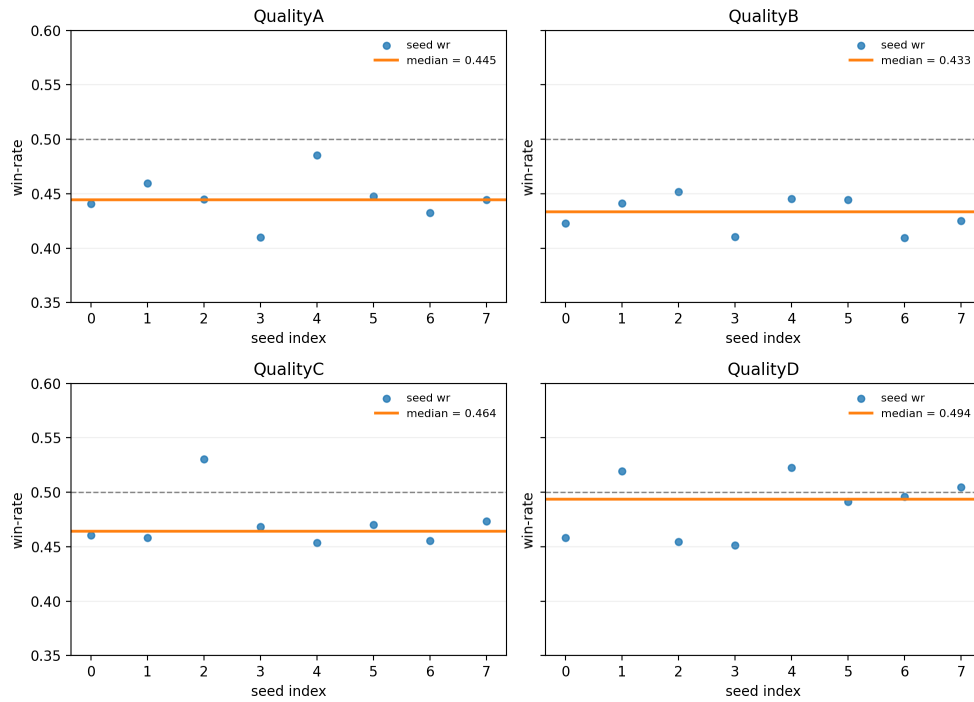


Figura 5.1. Tasso di vittoria finale in ES su ciascuno degli otto scenari di addestramento per i run QualityA–D. Ogni punto corrisponde a uno scenario; la linea arancione indica la mediana sugli scenari all'interno di un run e la linea orizzontale tratteggiata indica la soglia di training $\tau = 0.5$.

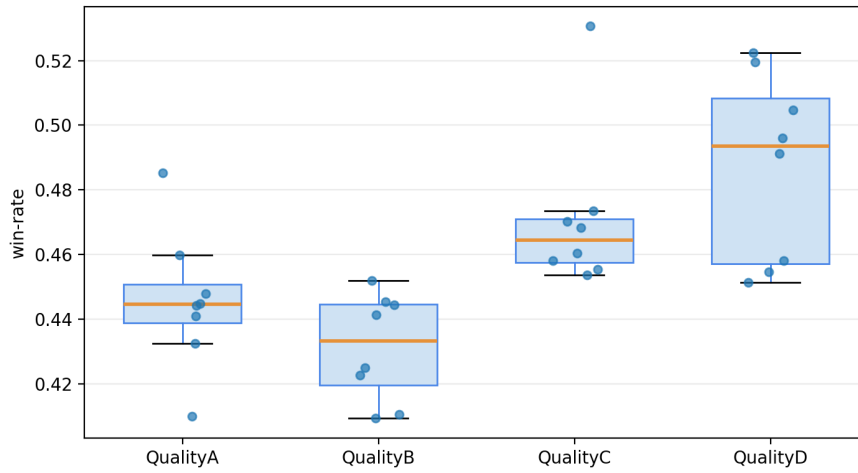


Figura 5.2. Distribuzione aggregata dei tassi di vittoria in ES per i run QualityA–D, mostrata come boxplot. Per ciascun run, il box copre il primo e il terzo quartile; il segmento arancione è la mediana; i baffi indicano il range e i punti sovrapposti rappresentano i singoli scenari.

5.2.1 Tassi di vittoria di training sull'insieme di scenari

Per un controllore θ fissato e uno scenario s , il tasso di vittoria di training empirico è definito come

$$\hat{p}(\theta, s) = \frac{\# \text{ vittorie su } s}{\# \text{ partite giocate su } s},$$

calcolato su un piccolo batch di partite di training. Le prestazioni complessive di training possono essere riassunte dal tasso di vittoria mediano sugli scenari $\tilde{p}$, dal tasso di vittoria peggiore $p_{\min}$ e dalla frazione di scenari che sembrano soddisfare il requisito alla risoluzione del training.

Sui quattro run QG (`logs_runQualityA-D`) con $|S| = 8$ scenari e soglia di training $\tau = 0.5$, emerge un pattern coerente. Tutti i run terminano con tassi di vittoria mediani per scenario compresi tra $0.433 \leq \tilde{p} \leq 0.494$, cioè vicini ma ancora leggermente inferiori alla soglia $\tau = 0.5$. Il tasso di vittoria peggiore resta tra 0.409 e 0.454, quindi nessun run riesce a soddisfare il vincolo di vittoria obbligata su *tutti* gli scenari. La frazione di scenari il cui valore di training supera τ cresce però tra i run: è pari a 0% per i run A e B, 12.5% per il run C e 37.5% per il run D. È importante notare che queste statistiche riassumono la distribuzione complessiva dei win-rate empirici sull'insieme degli scenari e non devono essere interpretate come indicatori di superamento o fallimento rispetto a una soglia. In particolare, sebbene il win-rate mediano e quello di caso peggiore risultino inferiori alla soglia obiettivo $\tau = 0.5$, una frazione non trascurabile di scenari raggiunge comunque win-rate empirici superiori a τ , come discusso di seguito. Questo conferma che ES migliora sistematicamente il controllore, ma mette anche in luce i limiti dell'architettura di rete e del budget di simulazione correnti: anche dopo l'ottimizzazione, il controllore rimane solo marginalmente al di sotto della specifica desiderata. Tendenze di questo tipo sono visibili nelle Figure 5.1 e 5.2.

5.3 Risultati della verifica scenario per scenario

Dopo il training ES, il controllore viene passato al verificatore sequenziale della Sezione 3.5. L'obiettivo è ottenere, per ciascun scenario $s \in S$:

- un tasso di vittoria empirico $\hat{p}_s$ calcolato con un numero di partite più elevato rispetto al training ES;
- un intervallo di confidenza di Wilson $[\ell_s, u_s]$ con copertura prestabilita;
- una decisione pass/fail rispetto al vincolo di vittoria obbligata con soglia $\tau = 0.6$.

Il verificatore salva i propri output in una tabella machine-readable con una riga per scenario, che include il numero totale di partite, la stima finale del tasso di vittoria, l'intervallo di Wilson e la classificazione pass/borderline/fail. Per i quattro run QG considerati, le statistiche risultanti sono riassunte in Tabella 5.1.

Run	N	Pass	Borderline	Fail	$p_{\min}$	$p_{\max}$
logs_runQualityA	8	0.0%	100.0%	0.0%	0.680	0.700
logs_runQualityB	8	0.0%	100.0%	0.0%	0.680	0.700
logs_runQualityC	8	0.0%	100.0%	0.0%	0.587	0.640
logs_runQualityD	8	0.0%	100.0%	0.0%	0.600	0.653

Tabella 5.1. Riepilogo della verifica sequenziale nei vari run QG (soglia $\tau = 0.6$). Per ciascun run, N è il numero di scenari in S , mentre $p_{\min}$ e $p_{\max}$ indicano i tassi di vittoria empirici minimo e massimo osservati al termine della verifica.

La Figura 5.3 mostra quante partite di verifica vengono giocate su ogni scenario nei vari run, mentre la Figura 5.4 visualizza i tassi di vittoria finali e gli intervalli di Wilson per ciascun run e scenario, con codifica cromatica della difficoltà.

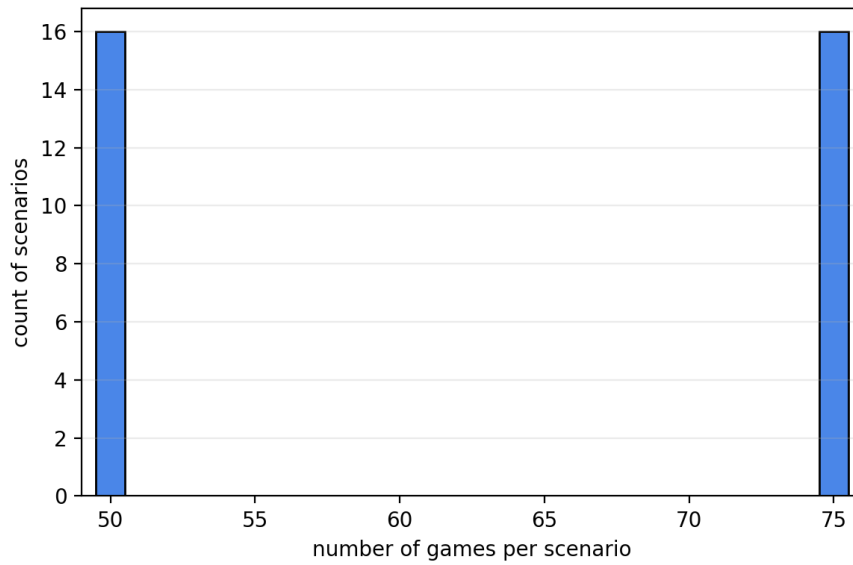


Figura 5.3. Distribuzione del numero totale di partite di verifica giocate per scenario nei run QualityA–D. Tutti gli scenari raggiungono il budget massimo consentito (50 o 75 partite, a seconda del run), riflettendo il fatto che tutti sono statisticamente ambigui al threshold $\tau = 0.6$.

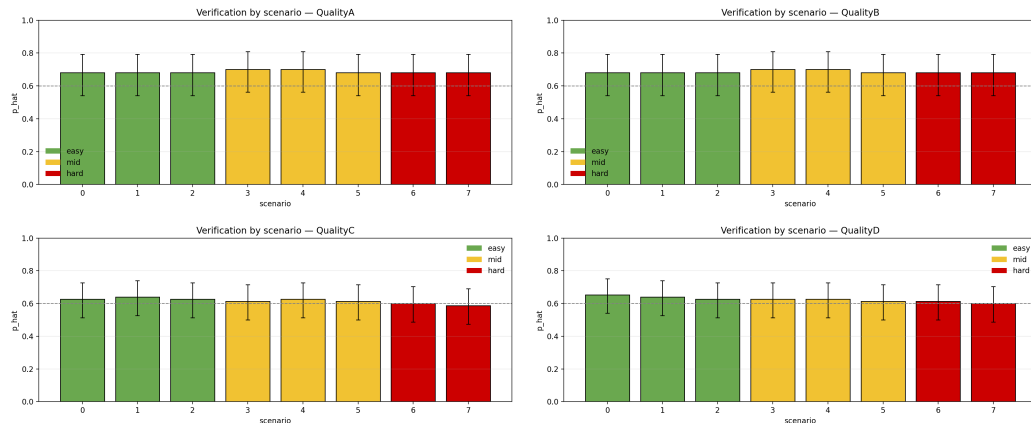


Figura 5.4. Risultati della verifica scenario per scenario per i run QualityA–D. Le barre mostrano i tassi di vittoria empirici $\hat{p}_s$ per ciascuno scenario, i segmenti verticali rappresentano gli intervalli di Wilson e i colori codificano la difficoltà dello scenario (easy, mid, hard). La linea tratteggiata orizzontale indica la soglia di verifica $\tau = 0.6$.

5.3.1 Dimensione dei campioni e ampiezza degli intervalli

La procedura sequenziale decide in modo adattivo quante partite giocare su ciascun scenario. Nei run QualityA e QualityB il numero massimo di partite per scenario è fissato a 50; nei run QualityC e QualityD questo limite viene alzato a 75. Come mostrato nella Figura 5.3, ogni scenario in ciascun run raggiunge il corrispondente limite, quindi gli intervalli di Wilson finali sono i più stretti possibili dato il budget.

Nonostante i limiti fissati, gli intervalli di Wilson risultanti sono ragionevolmente stretti. Per esempio, nei run QualityA e B i tassi di vittoria stimati si collocano nell'intervallo $[0.68, 0.70]$, chiaramente al di sopra della soglia di verifica $\tau = 0.6$ a livello di stime puntuali, sebbene gli intervalli di confidenza corrispondenti intersechino ancora τ a causa del numero finito di partite. Nei run QualityC e D l'intervallo si allarga a $[0.587, 0.640]$: alcuni scenari si trovano molto vicini alla soglia (con $\hat{p}_s \approx 0.60$), mentre altri sono più chiaramente al di sopra. Questo pattern è visibile nella Figura 5.4, in cui gli scenari più difficili (in rosso) tendono a concentrarsi attorno alla linea tratteggiata di soglia.

5.3.2 Decisioni pass/borderline/fail

Ogni scenario viene infine classificato in una delle tre categorie (cfr. Algoritmo 5):

- *pass* se il limite inferiore ℓ_s è chiaramente sopra τ ;
- *fail* se il limite superiore u_s è chiaramente sotto τ ;
- *borderline* se l'intervallo finale contiene τ ; in questo caso lo scenario è trattato in modo conservativo come fallito dal punto di vista QG.

Nel caso di studio sugli scacchi, tutti gli 8 scenari in ciascuno dei quattro run QG ricadono nella categoria *borderline*: non esistono scenari il cui limite inferiore sia chiaramente al di sopra di $\tau = 0.6$, né scenari il cui limite superiore sia chiaramente

al di sotto. Ciò è coerente con i valori in Tabella 5.1 e con la Figura 5.4, dove tutte le barre di errore intersecano la linea tratteggiata. Il verificatore certifica quindi che il controllore ottiene tassi di vittoria empirici attorno al 60%–70% sugli scenari di training, ma il budget disponibile non è sufficiente per dimostrare o confutare il vincolo di vittoria obbligata con il livello di confidenza desiderato, e tutti gli scenari vengono trattati in modo conservativo come potenziali violazioni nel ciclo QG.

Dal punto di vista statistico, questo esito rimane informativo. Quantifica con precisione quanto il controllore è vicino alla specifica e identifica gli scenari più critici (quelli con $\hat{p}_s$ appena sopra o appena sotto τ) come candidati naturali per ulteriori iterazioni QG o per modifiche architetturali al controllore.

5.4 Stime SMC della probabilità di violazione

La verifica scenario per scenario controlla le prestazioni sull'insieme finito S , ma non impone da sola un limite sulla probabilità di violazione $\gamma(\theta)$ sulla distribuzione di scenari più ampia D . Per questo, viene invocata la procedura SMC della Sezione 3.6.

5.4.1 Indicatori di violazione e intervalli di Wilson

Per ciascun scenario campionato $s^{(k)} \sim D$, SMC esegue un piccolo esperimento interno: viene giocato un numero fissato di partite sotto il controllore finale e il tasso di vittoria empirico $\hat{p}(\theta, s^{(k)})$ viene confrontato con τ . Questo produce un indicatore di violazione $Z_k \in \{0, 1\}$, trattato come una variabile di Bernoulli con media $\gamma(\theta)$.

Dopo N campioni, si calcola la stima empirica della probabilità di violazione $\hat{\gamma}_N = \frac{1}{N} \sum_{k=1}^N Z_k$ e il corrispondente intervallo di Wilson $[\ell_\gamma, u_\gamma]$. Come nel caso sequenziale scenario per scenario, il numero di campioni viene incrementato finché il raggio dell'intervallo non raggiunge l'accuratezza obiettivo o non si arriva a un massimo prestabilito.

La Figura 5.5 mostra la distribuzione dei tassi di vittoria SMC osservati nei quattro run QG, mentre la Figura 5.6 traccia la convergenza di $\hat{\gamma}_N$ e del relativo intervallo di confidenza.

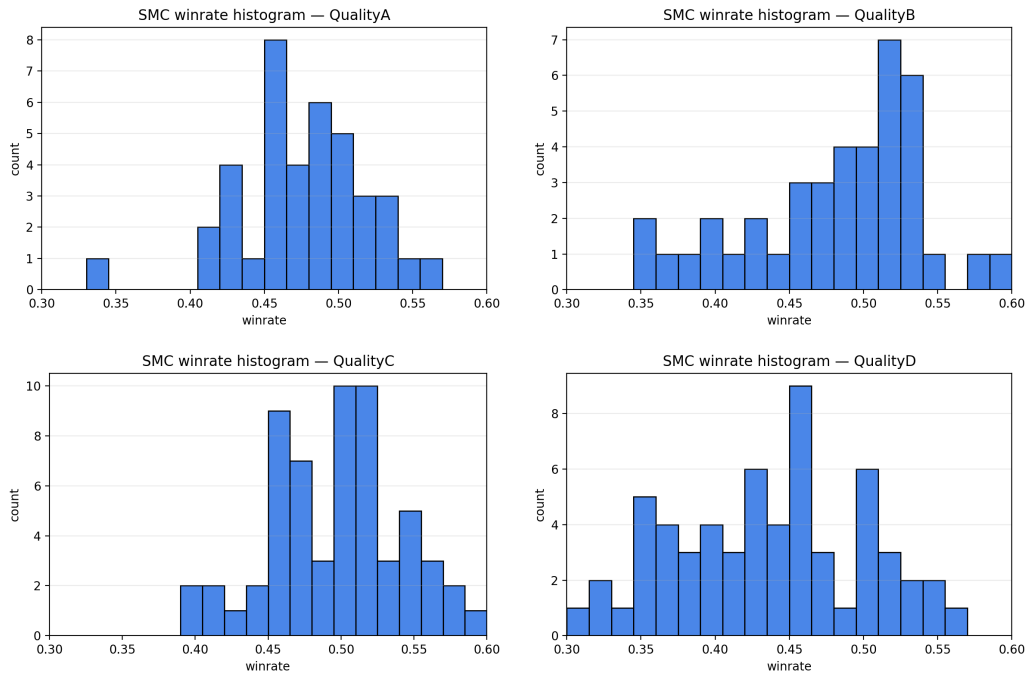


Figura 5.5. Istogrammi dei win-rate SMC sotto la distribuzione D per i run QualityA–D. In questi run, gli scenari campionati risultano al di sotto della soglia di verifica $\tau = 0.6$, producendo un indicatore di violazione $Z_k = 1$ per tutti i campioni SMC.

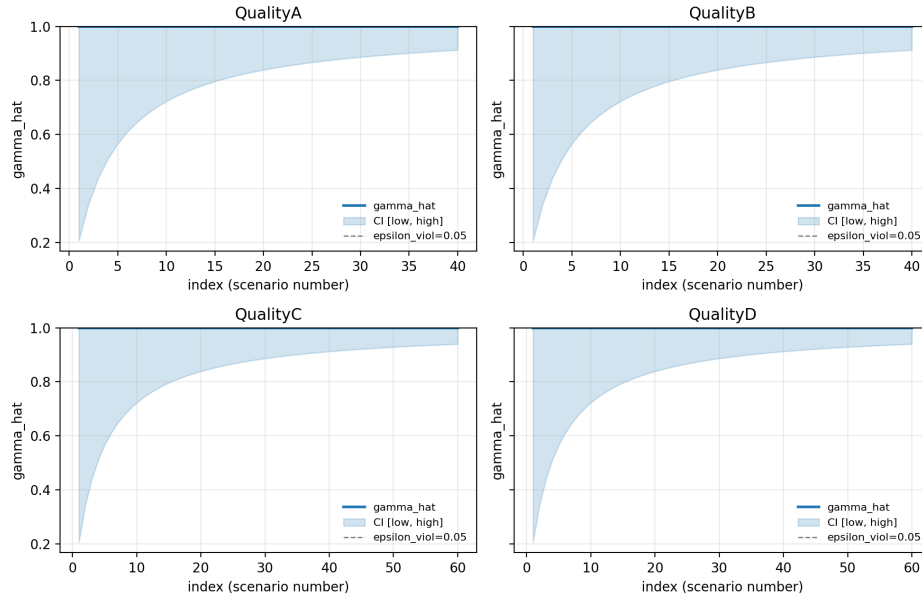


Figura 5.6. Convergenza della stima SMC della probabilità globale di violazione $\gamma(\theta)$ al crescere del numero di scenari campionati N . Sebbene l'asse orizzontale sia etichettato nel grafico come indice di scenario, esso corrisponde all'accumulazione sequenziale dei campioni SMC, ossia a valori crescenti di N . Le curve continue mostrano la stima empirica $\hat{\gamma}_N$ per i run QualityA–D, le regioni ombreggiate rappresentano gli intervalli di confidenza di Wilson e la linea orizzontale tratteggiata indica una tipica tolleranza QG $\varepsilon_{\text{viol}} = 0.05$.

In tutti e quattro i run QG utilizzati negli esperimenti sugli scacchi, ogni scenario campionato risulta violare il requisito. In altre parole, l'indicatore di violazione vale $Z_k = 1$ per tutti i campioni SMC. Di conseguenza, la stima $\hat{\gamma}_N$ rimane identicamente pari a 1 e l'intervallo di Wilson si concentra rapidamente vicino a 1 al crescere di N , come mostrato in Figura 5.6.

La Tabella 5.2 riporta le statistiche SMC finali per i quattro run.

Run	N	Violations	$\hat{\gamma}$	CI low	CI high	Radius
logs_runQualityA	40	40	1.000	0.912	1.000	0.044
logs_runQualityB	40	40	1.000	0.912	1.000	0.044
logs_runQualityC	60	60	1.000	0.940	1.000	0.030
logs_runQualityD	60	60	1.000	0.940	1.000	0.030

Tabella 5.2. Riepilogo della stima SMC globale. Per ciascun run, N è il numero di scenari SMC, $\hat{\gamma}$ è la probabilità di violazione empirica e [CI low, CI high] è l'intervallo di Wilson finale.

Nei run QualityA e B la procedura SMC si arresta dopo $N = 40$ scenari con un intervallo finale $[0.912, 1.000]$, corrispondente a un raggio di circa 0.044. Nei run QualityC e D, un budget maggiore di $N = 60$ scenari produce un intervallo ancora più stretto $[0.940, 1.000]$ con raggio 0.030. In tutti i casi, il limite inferiore è ben al

di sopra di qualunque tolleranza QG ragionevole $\varepsilon_{\text{viol}}$ (ad esempio 0.01 o 0.05), e l'algoritmo SMC certifica quindi che la probabilità di violazione globale è, di fatto, prossima a 1. In termini del criterio di arresto QG $u_\gamma \leq \varepsilon_{\text{viol}}$, questo implica che il ciclo non raggiunge mai una condizione di accettazione: con alta confidenza, il controllore corrente è lontano dal soddisfare il requisito sotto la distribuzione D .

Dal punto di vista QG, questo è un risultato negativo ma altamente informativo. Mostra che, mentre ES e la verifica scenario per scenario riescono a spingere gli scenari di training vicino alla soglia di vittoria obbligata, il controllore risultante non generalizza alla distribuzione più ampia D : sotto un campionamento realistico di seed e avversari, le violazioni sono ubiquitarie. La metodologia si comporta esattamente come previsto: invece di fornire un falso senso di sicurezza, si rifiuta di certificare un controllore la cui probabilità di violazione stimata è prossima a uno.

5.4.2 Natura degli scenari di violazione

La procedura SMC non solo stima $\gamma(\theta)$, ma raccoglie anche gli scenari di violazione come controesempi. L'ispezione di un sottoinsieme di questi scenari mostra che le violazioni raramente sono dovute a singoli errori grossolani in posizioni altrimenti banali. Tendono piuttosto a concentrarsi in situazioni strutturalmente difficili, come:

- aperture che producono squilibri fortemente sbilanciati di materiale o sicurezza del re in poche mosse;
- pattern tattici in cui è richiesta una breve sequenza accurata per evitare perdite di materiale;
- finali in cui il controllore deve giocare molte mosse precise per convertire un vantaggio a lungo termine.

Questo quadro qualitativo è coerente con i limiti di un controllore neurale poco profondo e privo di ricerca: senza un meccanismo di look-ahead, alcuni trappoloni tattici non possono essere rilevati dalla sola valutazione statica. Il fatto che SMC individui tali controesempi è una caratteristica, non un difetto: mostra che il ciclo QG è in grado di far emergere le regioni più problematiche dello spazio degli scenari invece di nasconderle dietro statistiche aggregate.

5.5 Budget complessivo di simulazione

La Tabella 5.3 riassume il numero di partite simulate utilizzate da ciascun componente del ciclo QG nei quattro run QG e nel run aggiuntivo con solo ES `logs_runRich2`. La Figura 5.7 fornisce una visualizzazione grafica degli stessi dati.

Run	ES games	Verification games	SMC games	Total
logs_runQualityA	800	400	400	1600
logs_runQualityB	800	400	400	1600
logs_runQualityC	2880	600	600	4080
logs_runQualityD	1728	600	600	2928
logs_runRich2	7500	750	750	9000
Grand total	13708	2750	2750	19208

Tabella 5.3. Numero di partite per run e per fase (ES, verifica sequenziale, SMC). L'ultima riga riporta i totali su tutti e cinque i run.

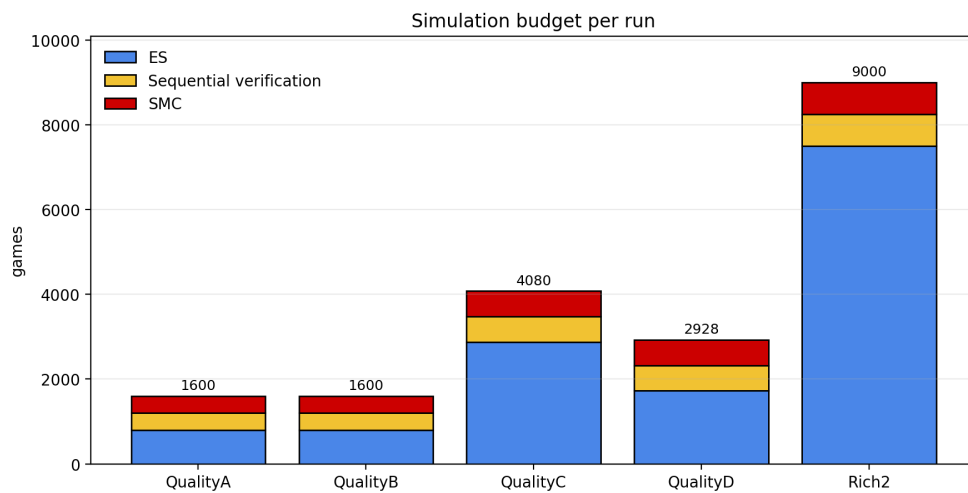


Figura 5.7. Grafico a barre impilate che mostra, per ciascun run, il numero di partite simulate utilizzate in ES, nella verifica sequenziale e in SMC. Le etichette sopra le barre indicano il numero totale di partite per run.

Complessivamente, gli esperimenti richiedono 19 208 partite simulate. Circa il 71.3% di esse (13 708 partite) vengono consumate da ES, riflettendo la natura di black-box del passo di ottimizzazione. La verifica sequenziale scenario per scenario e SMC globale assorbono ciascuna 2 750 partite, cioè circa il 14.3% del budget totale. Ciò conferma che i componenti statistici del ciclo QG aggiungono un overhead relativamente modesto rispetto allo sforzo di ottimizzazione, pur fornendo informazioni robuste sia sul comportamento scenario per scenario sia su quello globale.

Il run più ricco con solo ES `logs_runRich2` utilizza 7 500 partite per l'ottimizzazione e un numero moderato di partite aggiuntive per verifica e valutazione in stile SMC, illustrando come la stessa infrastruttura possa scalare a budget di ottimizzazione più ampi senza modificare il meccanismo statistico.

5.6 Comportamento a livello di partita e confronto con le baseline

Al di là dei tassi di vittoria aggregati e delle probabilità di violazione, è istruttivo osservare come il controllore addestrato si comporta a livello di singole partite e come si confronta con gli avversari di riferimento.

A tal fine, una versione iniziale del controllore (dopo un numero limitato di iterazioni ES) e una versione finale (dopo ES, verifica e SMC) vengono valutate in incontri diretti contro quattro avversari: `RandomPlayer`, `GreedyMaterialPlayer`, `PSQTGreedyPlayer` e `MinimaxPlayer`. Per ogni accoppiamento, viene giocato un grande batch di partite e si registrano le frequenze empiriche degli esiti e la lunghezza mediana delle partite.

5.6.1 Partite contro avversari semplici

Contro l'avversario random, il controllore mostra il comportamento atteso di una politica ragionevolmente addestrata. La Tabella 5.4 mostra che la versione iniziale del controllore vince il 63.2% delle partite, pattando il 10.3% e perdendo il 26.5%, con una lunghezza mediana delle partite di 180 mosse (le mezze mosse sono contate come mosse nei log). La versione finale migliora ulteriormente il tasso di vittoria al 73.6%, riduce il tasso di patta al 3.5% e quello di sconfitta al 22.9% e chiude le partite più rapidamente, con una mediana di 150 mosse.

Questi numeri indicano che ES e il ciclo QG riescono a produrre un controllore in grado di sfruttare in modo affidabile le debolezze di una politica completamente casuale e di convertire i vantaggi in maniera abbastanza decisiva. La riduzione della lunghezza mediana suggerisce che il controllore finale è più efficiente nel capitalizzare le opportunità tattiche quando si presentano.

5.6.2 Partite contro baseline più forti

Il quadro cambia sensibilmente passando a baseline più forti. Contro `GreedyMaterialPlayer` e `MinimaxPlayer`, sia la versione iniziale sia quella finale del controllore non riescono a ottenere nemmeno una vittoria: in tutti i casi il tasso di vittoria empirico è 0%, mentre le patte rappresentano circa l'83% delle partite e le sconfitte il 15%–18%, con lunghezze mediane attorno alle 72–77 mosse. Contro `PSQTGreedyPlayer` la situazione è ancora più impegnativa: anche qui il controllore non vince mai, e il tasso di sconfitta aumenta dal 26.0% (fase iniziale) al 39.8% (fase finale), con lunghezze mediane delle partite molto elevate, attorno a 112–114 mosse.

Questo comportamento riflette i limiti di un controllore neurale poco profondo e privo di ricerca. Contro avversari che sfruttano anche solo un minimo look-ahead o una valutazione statica più raffinata (come nel caso PSQT), la rete riesce spesso a mantenere una posizione approssimativamente equilibrata per molte mosse, ma fatica a trasformare piccoli vantaggi in vittorie. La versione finale del controllore è più decisiva contro il gioco random ma non riesce a superare in modo sistematico la difesa delle baseline basate su ricerca.

5.6.3 Statistiche quantitative a livello di partita

La Tabella 5.4 riassume i risultati a livello di partita riportando, per ciascun avversario, le frequenze empiriche degli esiti e la lunghezza mediana delle partite per la versione iniziale e quella finale del controllore.

Opponent	Early W	Early D	Early L	Late W	Late D	Late L
GreedyMaterialPlayer	0.0%	82.8%	17.2%	0.0%	82.1%	17.9%
MinimaxPlayer	0.0%	83.9%	16.1%	0.0%	85.1%	14.9%
PSQTGreedyPlayer	0.0%	74.0%	26.0%	0.0%	60.2%	39.8%
RandomPlayer	63.2%	10.3%	26.5%	73.6%	3.5%	22.9%

Tabella 5.4. Esiti di partita (early vs. late). Le percentuali di vittorie, patte e sconfitte sono calcolate sullo stesso numero di partite per ciascuna coppia controllore–avversario.

Il confronto tra la versione iniziale e quella finale del controllore mostra un effetto qualitativo chiaro del ciclo QG. Contro il baseline random, il controllore diventa sia più forte sia più decisivo: il tasso di vittoria aumenta di oltre 10 punti percentuali, le patte diventano rare e le partite tendono a concludersi prima. Questo è coerente con l'intuizione che un training ES orientato alla fattibilità insegna alla rete a sfruttare in modo più affidabile gli errori tattici ovvi e a convertire i vantaggi materiali.

Contro le baseline basate su ricerca, invece, la storia è diversa. La versione finale del controllore non riesce a trasformare posizioni pari in vittorie e, nel caso di **PSQT-GreedyPlayer**, perde persino più spesso della versione iniziale. Dal punto di vista QG, questo è perfettamente coerente con il verdetto negativo di SMC (Tabella 5.2): il controllore può apparire competente in molte partite, ma lo spazio degli scenari realistici contiene una grande frazione di posizioni in cui la sola valutazione statica non basta ad evitare violazioni del vincolo di vittoria obbligata.

Nel complesso, l'analisi a livello di partita conferma che il ciclo QG è informativo oltre i tassi di vittoria aggregati: mette in luce, attraverso statistiche concrete, il divario tra “giocare in modo plausibile” e “soddisfare la specifica in modo dimostrabile”.

5.7 Sintesi dei risultati sperimentali

Gli esperimenti riportati in questo capitolo rispondono alle principali domande poste nell'introduzione della tesi e illustrano sia i punti di forza sia i limiti attuali del ciclo QG quando applicato a un gioco non banale come gli scacchi.

1. **Il training orientato alla fattibilità produce miglioramenti sistematici ma non soddisfa pienamente la specifica.** ES, accoppiato con la perdita di fattibilità $J_S(\theta)$, è in grado di trovare controllori neurali che migliorano sensibilmente rispetto alla politica iniziale e raggiungono tassi di vittoria mediani sugli scenari tra il 43% e il 49% alla soglia $\tau = 0.5$, come mostrato nelle Figure 5.1 e 5.2. Tuttavia, anche il miglior run lascia alcuni scenari al di sotto del vincolo di vittoria obbligata e nessun run ottiene un controllore pienamente fattibile sull'insieme di training.

2. **La verifica sequenziale fornisce una quantificazione esplicita dell'incertezza scenario per scenario.** Il verificatore basato su intervalli di Wilson concentra i campioni sugli scenari più ambigui e produce intervalli di confidenza per scenario (Tabella 5.1, Figure 5.3 e 5.4). Nel caso di studio corrente, tutti gli scenari risultano borderline a $\tau = 0.6$, indicando che tassi di vittoria empirici nell'intervallo 0.59–0.70 non sono sufficienti, dato il budget disponibile, per dimostrare o confutare il vincolo di vittoria obbligata. Si tratta di un esito negativo dal punto di vista delle garanzie, ma positivo dal punto di vista modellistico: trasforma l'idea informale di “quasi abbastanza buono” in un'affermazione quantitativa.
3. **SMC mette in luce la mancanza di robustezza globale.** Campionando scenari da D e stimando $\gamma(\theta)$ con controllo di errore assoluto, SMC colma il divario tra l'insieme di training finito e lo spazio di scenari più ampio che il controllore può incontrare. In tutti i run QG, la probabilità di violazione stimata converge a $\hat{\gamma} = 1.0$ con intervalli di Wilson stretti, come $[0.940, 1.000]$ (Tabella 5.2). Questo mostra che il controllore corrente è lontano dal soddisfare qualsiasi tolleranza QG realistica e che sono necessarie ulteriori modifiche architetturali o strategie di training per ottenere garanzie non banali.
4. **I costi di simulazione rimangono moderati e contabilizzati in modo trasparente.** L'intera campagna sperimentale utilizza 19 208 partite simulate, di cui circa il 71% è dedicato a ES e il restante 29% è diviso in parti uguali tra verifica sequenziale e SMC (Tabella 5.3, Figura 5.7). Ciò conferma che la componente statistica del ciclo QG può essere applicata con un overhead modesto rispetto a pipeline di ottimizzazione black-box già esistenti.
5. **Il controllore risultante gioca partite plausibili ma è significativamente più debole di semplici baseline basate su ricerca.** Gli esperimenti a livello di partita mostrano che il controllore supera nettamente un avversario random e migliora il proprio tasso di vittoria e la propria capacità di chiudere le partite dopo il ciclo QG, ma non riesce a ottenere vittorie contro baseline basate su ricerca anche modeste e perde una frazione significativa di partite (Tabella 5.4). Ciò è coerente con il verdetto negativo di SMC e sottolinea che, nella forma attuale, il controllore non dovrebbe essere utilizzato in contesti safety-critical senza ulteriori affinamenti.

Nel complesso, questi risultati convalidano il framework QG come strumento pratico per analizzare controllori basati su apprendimento: anche quando il verdetto finale è negativo (cioè i requisiti non sono soddisfatti), la combinazione di ES, verifica sequenziale e SMC produce un quadro dettagliato e statisticamente fondato di dove e come il controllore fallisce, e a quale costo computazionale. In lavori futuri, la stessa metodologia potrà essere applicata ad architetture di rete più potenti e a modelli di avversario più ricchi, con l'obiettivo di trasformare intuizioni informali su “giocare bene” in garanzie di qualità esplicite.

Sopra i componenti core, una semplice interfaccia grafica costruita sullo stesso motore e controllore permette di ispezionare le partite loggate e di giocare interattivamente contro il controllore addestrato. Sebbene non sia essenziale per la

metodologia QG in sé, questa interfaccia risulta utile per controlli di coerenza: offre una vista umanamente leggibile del comportamento appreso e aiuta a confermare che le garanzie teoriche siano riferite a un controllore che gioca partite legali e riconoscibilmente “in stile scacchistico”.

Capitolo 6

Conclusioni

Questa tesi ha studiato la progettazione basata sulla simulazione di sistemi intelligenti con garanzie di qualità esplicite, istanziando il ciclo di apprendimento Quality-Guaranteed (QG) in un dominio avversariale non banale: gli scacchi. La pipeline proposta combina Evolution Strategies (ES), verifica sequenziale scenario per scenario, Statistical Model Checking (SMC) e raffinamento guidato da controesempi in un unico ciclo chiuso che ottimizza un controllore neurale stimandone simultaneamente la probabilità di violare un requisito di tipo must-win.

Questo capitolo riassume i risultati principali, risponde alle domande di ricerca introdotte nel Capitolo 1, discute i limiti dell'attuale caso di studio e delinea alcune direzioni per lavori futuri.

6.1 Sintesi della tesi

Il lavoro parte dall'osservazione che la maggior parte delle pipeline di machine learning ottimizza la prestazione media, ma offre scarso supporto per garanzie quantitative del tipo “il sistema vince con probabilità almeno τ e confidenza $1 - \delta$ ”. Per colmare questo divario, la tesi adotta il paradigma QG: l'apprendimento è guidato da un requisito ed è strettamente accoppiato alla verifica statistica.

Dal punto di vista metodologico, la tesi:

- formalizza l'ambiente degli scacchi come un modello di simulazione controllato con scenari riproducibili, costruito sopra il motore `GameState`;
- definisce un controllore neurale feed-forward leggero $f_\theta(x)$ e un requisito espresso in termini di soglie di win-rate su una distribuzione di scenari;
- adatta ES a un training orientato alla fattibilità, in cui l'obiettivo è minimizzare una loss che penalizza la violazione del requisito sul win-rate;
- sviluppa un verificatore sequenziale che, per ciascuno scenario, adatta il numero di partite simulate e restituisce intervalli di confidenza di Wilson e decisioni pass/borderline/fail a un livello di confidenza prefissato;
- integra una procedura SMC globale che stima la probabilità di violazione empirica $\gamma_{\text{int}}(\theta)$ sotto una distribuzione di scenari non visti, ancora una volta con intervalli di tipo Wilson e controllo esplicito dell'errore;

- implementa l'intero ciclo QG in Python, includendo gestione di scenari e controesempi, logging e strumenti per l'analisi a livello di partita.

Dal punto di vista sperimentale, la tesi applica questa pipeline a una famiglia di run QG in cui il controllore viene addestrato e raffinato contro un piccolo insieme fisso di scenari e successivamente valutato tramite SMC su una distribuzione di scenari più ampia. L'analisi del Capitolo 5 quantifica:

- come ES migliori i win-rate mediani e quelli di caso peggiore sull'insieme di training senza soddisfare completamente il requisito must-win;
- come la verifica sequenziale concentri il proprio sforzo sugli scenari statisticamente ambigui e fornisca intervalli di confidenza scenario per scenario;
- come lo SMC mostri che, nonostante i miglioramenti locali, la probabilità di violazione empirica sia essenzialmente pari a 1, mettendo in luce una mancanza di robustezza fuori dall'insieme di training;
- come il budget complessivo di simulazione sia distribuito tra ES, verifica e SMC, e come tale budget rimanga moderato rispetto alla scala del caso di studio;
- come il controllore addestrato si comporti a livello di partita contro avversari random e avversari basati su ricerca, evidenziando il divario tra gioco "plausibile" e soddisfazione reale del requisito.

6.2 Risposte alle domande di ricerca

RQ1. Un controllore neurale leggero addestrato tramite ES può soddisfare un requisito must-win su un insieme finito di scenari?

Gli esperimenti mostrano che un training ES orientato alla fattibilità è in grado di migliorare in modo sostanziale il controllore sull'insieme finito di scenari. Nei vari run QG, i win-rate mediani per scenario si avvicinano alla soglia di training e la frazione di scenari il cui win-rate empirico supera la soglia aumenta nel tempo. Tuttavia, nessun run ottiene un controllore che soddisfi il requisito must-win su tutti gli scenari di training.

In altre parole, ES è efficace nello spingere il controllore verso la fattibilità, ma l'architettura scelta e il budget disponibile non sono sufficienti a superare il confine tra "quasi abbastanza buono" e il soddisfare dimostrabilmente il requisito. Dal punto di vista della progettazione del controllore, questo indica che ES orientato alla fattibilità può fungere da forte dorsale di ottimizzazione, ma il suo successo dipende in ultima analisi dal potere espressivo della classe di controllori e dal budget di simulazione disponibile.

RQ2. La verifica sequenziale e lo SMC possono fornire garanzie significative entro budget di simulazione realistici?

La verifica sequenziale e lo SMC si dimostrano al tempo stesso informativi ed efficienti. Sugli scenari di training, il verificatore sequenziale seleziona in modo adattivo le

dimensioni campionarie fino a un tetto massimo per scenario e restituisce intervalli di Wilson la cui ampiezza è coerente con il budget. Nel caso di studio sugli scacchi, il verificatore classifica tutti gli scenari come statisticamente borderline rispetto alla soglia must-win: i win-rate empirici sono vicini alla soglia, ma gli intervalli di confidenza non consentono un verdetto definitivo pass/fail. Questo esito negativo è comunque prezioso, perché trasforma affermazioni qualitative del tipo “il controllore sembra vincere la maggior parte delle volte” in limiti quantitativi sulle probabilità sottostanti.

A livello globale, lo SMC stima la probabilità di violazione empirica $\gamma_{n_{\text{int}}}(\theta)$ sotto una distribuzione di scenari che campiona seed e avversari. Per tutti i run QG, la procedura SMC produce intervalli di Wilson stretti, il cui limite inferiore è vicino a 1, mostrando che le violazioni sono ubiquitarie sotto un campionamento realistico. Nonostante questo verdetto pessimista, il numero richiesto di partite simulate rimane modesto rispetto al budget di ES, e l'intera pipeline si colloca nell'ordine di poche decine di migliaia di partite. Ciò conferma che, almeno per domini di simulazione di complessità paragonabile agli scacchi, garanzie in stile QG possono essere ottenute entro risorse computazionali realistiche.

RQ3. Il raffinamento guidato da controesempi migliora la robustezza rispetto a scenari non visti?

Il meccanismo dei controesempi raccoglie gli scenari falliti dalla verifica sequenziale e dallo SMC e li reinserisce nell'insieme di training. In linea di principio, questo processo dovrebbe concentrare ES sulle regioni più problematiche dello spazio degli scenari e ridurre la probabilità di violazione stimata lungo successive iterazioni QG.

Nel presente caso di studio, i risultati sono misti. Da un lato, gli scenari aggiunti arricchiscono effettivamente l'insieme di training e rendono la loss di fattibilità più severa, come mostra l'evoluzione dei win-rate di training nei diversi run. Dall'altro lato, lo SMC riporta in modo sistematico una probabilità di violazione prossima a 1, indicando che, data l'attuale classe di controllori e il budget disponibile, il ciclo QG non riesce a portare il sistema in un regime di violazione realmente bassa.

La lezione principale è che il raffinamento guidato da controesempi è efficace nel *far emergere* le modalità di fallimento — rende visibili situazioni strutturalmente difficili, come trappole tattiche acute o lunghi finali forzanti — ma non può, da solo, compensare una capacità di modello insufficiente o un requisito troppo ambizioso. In questo senso, il ciclo QG si comporta come uno strumento diagnostico adattivo: quando fallisce, fallisce con spiegazioni.

6.3 Limiti del caso di studio

Le conclusioni sopra esposte dipendono da diverse scelte di modellazione e da alcune semplificazioni.

Architettura del controllore. Il controllore neurale è intenzionalmente poco profondo e compatto, scelto per velocità e interpretabilità più che per pura forza di gioco. Questo rende più semplice attribuire i fallimenti alla mancanza di look-ahead

o di capacità rappresentativa, ma limita anche le prestazioni raggiungibili. Il verdetto negativo dello SMC non va quindi interpretato come un limite fondamentale del controllo QG, bensì come una conseguenza di questa scelta progettuale.

Distribuzione di scenari e requisito. Il requisito è definito su una specifica distribuzione di scenari che combina diversi avversari e seed pseudo-casuali. Le garanzie, così come le probabilità di violazione, sono significative solo rispetto a tale distribuzione. Modificare il mix di avversari, i controlli di tempo o il campionamento dei seed condurrebbe a conclusioni numeriche diverse. Inoltre, la soglia must-win e la tolleranza adottate sono intenzionalmente severe: in molte applicazioni pratiche, requisiti più deboli (ad esempio su regret atteso o su utilità sensibili al rischio) potrebbero risultare più appropriati.

Scala e budget computazionale. Il numero totale di partite simulate, pur non trascurabile, è ridotto rispetto alle moderne pipeline di reinforcement learning nei giochi da tavolo. Il caso di studio va quindi interpretato come una prova di concetto con budget vincolato, non come un tentativo di competere con i motori scacchistici allo stato dell'arte. Scalare i metodi QG a controllori più grandi e spazi di scenari più ricchi richiederà ulteriore lavoro di ingegnerizzazione, parallelizzazione e un attento tuning dei criteri di arresto.

Specificità del dominio. Gli scacchi sono deterministici e completamente osservabili, con un simulatore ben definito e senza implicazioni dirette di sicurezza nel mondo reale. Queste proprietà li rendono un banco di prova ideale per il lavoro metodologico, ma eliminano anche diverse sfide che emergono nei sistemi cyber-fisici, come l'incertezza di modello, il rumore sui sensori o le dinamiche miste continuo-discrete. Estrarre dai risultati quantitativi di questa tesi conclusioni dirette su tali domini sarebbe ingiustificato; ciò che si trasferisce è la *struttura* del ciclo QG.

6.4 Prospettive e lavori futuri

L'evidenza sperimentale raccolta in questa tesi suggerisce diverse direzioni per ulteriori ricerche.

Controllori più forti e avversari più ricchi

Un'estensione naturale consiste nell'aumentare il potere espressivo del controllore, ad esempio usando reti più profonde, architetture convoluzionali o teste separate di policy e value. Accoppiare tali controllori con meccanismi di ricerca (ad esempio una Monte Carlo tree search che usi f_θ come funzione di valutazione) permetterebbe inoltre di testare come i metodi QG interagiscano con meccanismi di look-ahead. Dal lato degli avversari, introdurre baseline più forti e stili di gioco più diversificati renderebbe la distribuzione di scenari più impegnativa e fornirebbe un quadro più fine della robustezza.

Requisiti alternativi e progettazione di controllo multi-obiettivo

Il requisito attuale è definito come un semplice vincolo must-win sul win-rate. In molte applicazioni, si vorrebbe invece bilanciare prestazione media, rischio di fallimenti catastrofici e costo computazionale. Estendere il framework QG a requisiti multi-obiettivo o gerarchici — ad esempio combinando vincoli su win-rate, probabilità massima di perdita e uso delle risorse — lo renderebbe più vicino alla pratica ingegneristica reale. Dal punto di vista statistico, questo richiederebbe enunciati di confidenza congiunti su più metriche.

Miglioramento della componente statistica

Sul fronte della verifica, c'è spazio per metodi statistici più avanzati. Test sequenziali con regole di arresto più strette, tecniche di riduzione della varianza o intervalli credibili bayesiani potrebbero ridurre il numero di simulazioni richieste mantenendo garanzie rigorose. Un'allocazione adattiva del budget di simulazione tra scenari, basata sulla loro difficoltà stimata, potrebbe aumentare ulteriormente l'efficienza. Esplorare queste opzioni in combinazione con ES e raffinamento guidato da controesempi rappresenta una linea di lavoro promettente.

Applicazioni oltre gli scacchi

Forse la prospettiva più importante è trasferire le lezioni di questo caso di studio a domini in cui le garanzie quantitative sono direttamente legate alla sicurezza o all'affidabilità. Esempi includono il motion planning per veicoli autonomi, la gestione delle risorse nelle reti di comunicazione o il controllo supervisore nell'automazione industriale. In contesti di questo tipo, il controllo QG potrebbe fungere da ponte tra apprendimento guidato dai dati e verifica tradizionale, fornendo sia controllori ad alte prestazioni sia limiti superiori espliciti sulla probabilità di fallimento.

Osservazioni finali

Il caso di studio sugli scacchi dimostra che un ciclo di apprendimento Quality-Guaranteed può essere istanziato end-to-end in un dominio complesso e avversariale, producendo non solo un controllore neurale addestrato ma anche un ricco insieme di diagnostiche statistiche sul suo comportamento. Il verdetto finale in questo specifico esperimento è negativo: il controllore non soddisfa il requisito must-win sotto la distribuzione di scenari scelta. Tuttavia, anche questo risultato negativo è informativo, perché è supportato da espliciti limiti di confidenza ed è accompagnato da controesempi concreti.

In questo senso, il contributo principale della tesi è concettuale più che competitivo: mostra che apprendimento e verifica non devono necessariamente essere fasi separate, e che l'ottimizzazione basata sulla simulazione può essere inserita in un ciclo che testa continuamente, spiega e, quando possibile, garantisce il comportamento di un sistema intelligente.

Ringraziamenti

Desidero innanzitutto ringraziare me stesso per aver portato a termine questo percorso. Per la pazienza, per la forza con cui ho affrontato le montagne incontrate lungo la strada e per il coraggio di perdonare i miei errori, riconoscendoli come parte di chi sono stato e di chi diventerò. Sono grato al me del passato per aver scelto la perseveranza invece della comodità, e spero che il me del futuro guardi a questo momento con la stessa determinazione e con un quieto orgoglio. Mi ringrazio per non essermi arreso nei momenti in cui sarebbe stato più facile fermarsi, per aver continuato a studiare, a migliorarmi e a credere che questo traguardo fosse alla mia portata.

Vorrei ringraziare la mia famiglia, che mi ha sostenuto non solo durante questo percorso universitario, ma fin dal primo giorno della mia vita. Mi hanno dato l'opportunità di ricevere una buona istruzione e, ancora più importante, mi hanno aiutato a capire quanto la conoscenza sia essenziale nella vita. Tra tutte le persone, desidero dedicare alcune righe a mia madre, Cornelia Banua Trinidad: senza di lei non sarei la persona che sono oggi. È stata la mia fonte di luce in ogni momento buio e mi ha permesso di coltivare le mie passioni, dandomi al tempo stesso la libertà di esplorare il mondo a modo mio. La ringrazio per i suoi sacrifici silenziosi, per aver creduto nel mio futuro anche quando io stesso avevo dei dubbi e per avermi insegnato, con l'esempio, che cosa significhi davvero dedicarsi a chi si ama.

Un ringraziamento speciale va anche alle mie maestre della scuola primaria, Maria Grazia Bultrini e Francesca Diaco, che mi hanno accompagnato nei primi passi del mio percorso accademico e hanno creduto in me fin dall'inizio. A loro devo le basi del mio cammino, ma anche qualcosa di ancora più prezioso: la curiosità per lo studio, la pazienza di riprovare dopo gli errori e la consapevolezza che, con perseveranza e impegno, si possa davvero imparare qualsiasi cosa.

Un pensiero di gratitudine va ai miei amici più stretti — Rita Rotondaro, Greta Quarta, Aurora Parlato, Laura Stefani, Federico Rizzo, Matteo Greco, Alfonso De Cesare e Luca Carioscia — che mi hanno accompagnato in questo viaggio. Grazie per aver condiviso con me sia i giorni più leggeri sia quelli più difficili, per le risate, il sostegno e la vostra presenza costante. Ognuno di voi ha reso questo cammino meno pesante e infinitamente più significativo.

Un ringraziamento speciale va anche ai miei colleghi universitari — Alberto Pulcini, Daniele Marchetilli, Daniele Spadea e Bernardo Marchese — con cui ho condiviso non solo lezioni ed esami, ma anche dubbi, ambizioni e innumerevoli conversazioni. Crescere insieme a voi ha reso questo percorso intellettualmente stimolante e umanamente arricchente. Grazie per i confronti, per il sostegno reciproco e per aver trasformato le difficoltà in esperienze condivise.

Alla professoressa Giulia Latini, che mi ha accompagnato fin dall'inizio del mio cammino. Grazie per la cura, la pazienza e l'affetto con cui mi ha sempre guidato. Il suo esempio resterà sempre parte del mio percorso.

Vorrei inoltre ringraziare Dio, se esiste: mi piace pensare che, da qualche parte, stia sorridendo. Spero che, se davvero sta osservando, possa essere almeno un po' orgoglioso del cammino che sto percorrendo e che continui a darmi la lucidità e la forza di cui ho bisogno per inseguire i miei sogni.

Questa tesi non rappresenta una conclusione, ma un inizio. È un promemoria del fatto che la crescita avviene spesso in silenzio — attraverso sforzi ripetuti, dubbi affrontati e piccoli passi compiuti con costanza nel tempo. Rappresenta non solo ciò che ho imparato finora, ma anche la responsabilità di continuare a imparare, a mettermi in discussione e a costruire sistemi — e una vita — che riflettano i valori in cui scelgo di credere. Qualunque cosa venga dopo, vado avanti con gratitudine, umiltà e con la quieta determinazione che mi ha portato fin qui.

E all'inizio di questo cammino — grazie per aver avuto il coraggio di iniziare.

Bibliografia

- [1] Petr Bulychhev, Alexandre David, Kim G. Larsen, Axel Legay, Marius Mikucionis, Danny Bøgsted Poulsen, and Zheng Wang. Uppaal-smc: Statistical model checking for priced timed automata. In *Proceedings of the 10th Workshop on Quantitative Aspects of Programming Languages and Systems (QAPL 2012)*, volume 85 of *Electronic Proceedings in Theoretical Computer Science*, pages 1–16. Open Publishing Association, 2012.
- [2] Alexandre David, Dehui Du, Kim G. Larsen, Axel Legay, Marius Mikucionis, Danny Bøgsted Poulsen, and Sean Sedwards. Statistical model checking for stochastic hybrid systems. In *Proceedings of the 1st International Workshop on Hybrid Systems and Biology (HSB 2012)*, volume 92 of *Electronic Proceedings in Theoretical Computer Science*, pages 122–136. Open Publishing Association, 2012.
- [3] Marco Esposito, Alberto Leva, Toni Mancini, Leonardo Picchiami, and Enrico Tronci. Simulation-based design of industry-size control systems with formal quality guarantees. *IEEE Transactions on Industrial Informatics*, 21(5):3871–3879, 2025.
- [4] Niklas Fiekas. python-chess: A chess library for Python. <https://python-chess.readthedocs.io/>, 2024.
- [5] Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975.
- [6] Ingo Rechenberg. *Evolutionsstrategie: Optimierung technischer Systeme nach Prinzipien der biologischen Evolution*. Frommann-Holzboog, Stuttgart, 1973.
- [7] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.
- [8] Hans-Paul Schwefel. *Numerische Optimierung von Computer-Modellen mittels der Evolutionsstrategie*. Birkhäuser, Basel, 1977. In German.
- [9] Koushik Sen, Mahesh Viswanathan, and Gul Agha. Statistical model checking of black-box probabilistic systems. In *Computer Aided Verification (CAV 2005)*, volume 3576 of *Lecture Notes in Computer Science*, pages 202–215. Springer, 2005.

- [10] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.
- [11] Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley, 2008.
- [12] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [13] Abraham Wald. *Sequential Analysis*. Wiley, 1947.
- [14] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [15] Håkan L. S. Younes. Probabilistic verification of discrete event systems using acceptance sampling. In *Computer Aided Verification (CAV 2004)*, volume 3114 of *Lecture Notes in Computer Science*, pages 223–235. Springer, 2004.